

POST-QUANTUM CRYPTOGRAPHY: IMPLEMENTATION OF A HYBRID HANDSHAKE

Sergi Rivet Fernandez



Universitat
Pompeu Fabra
Barcelona

POST-QUANTUM CRYPTOGRAPHY: IMPLEMENTATION OF A HYBRID HANDSHAKE

BACHELOR'S THESIS

Sergi Rivet Fernandez

Supervisor: Carla Ràfols

Bachelor's Degree in Computer Engineering

Academic Year 2025-2026



Universitat
Pompeu Fabra
Barcelona

Escola
d'Enginyeria

Acknowledgements

This final year has been a challenging time for me, filled with obstacles that pushed my limits. However, it has also been a journey of transformation that helped me figure out what I truly want in life. Looking back, all the effort, sacrifice, and sleepless nights have been completely worth it. This experience has taught me the important lesson of trusting my own decisions and instincts while staying true to my path, no matter what others think or say.

I could not have gotten through this without the constant encouragement, patience, and support of my family and friends, to whom I am deeply grateful. I also want to give special recognition to those who are no longer with us; even though they cannot see the end of this chapter, their memory and light have been my quiet strength every day.

I am also very thankful to my thesis advisor, Carla Ràfols, for her invaluable guidance and support during the development of this project.

Finally, this work has been funded by the Artemisa Chair, an initiative carried out within the framework of the funds of the Recovery, Transformation and Resilience Plan, financed by the European Union (Next Generation), the project of the Spanish Government that traces the roadmap for the modernization of the Spanish economy, the recovery of economic growth and job creation, for the solid, inclusive and resilient economic reconstruction after the COVID-19 crisis, and to respond the challenges of the next decade.

Summary

This thesis investigates the transition towards cryptographic resilience against quantum computing. Current public-key cryptographic systems, including RSA and ECC, whose security is based on the intractability of integer factorization and the Discrete Logarithm Problem (DLP), are compromised by Shor's algorithm. Symmetric-key primitives are also affected by quantum computing through Grover's algorithm, which forces a length increase of security parameters since it reduces their security level.

To mitigate this vulnerability, this work analyses lattice-based Post-Quantum Cryptography (PQC) primitives based on the hardness of both Learning With Errors (LWE) and Short Integer Solution (SIS) problems, specifically focusing on the standardized implementations such as ML-KEM and ML-DSA.

Finally, a Hybrid Authenticated Handshake Protocol is designed, integrating both X25519 and ML-KEM via HKDF while incorporating nonces, composite signatures, and HMAC for integrity. The benchmark measurements demonstrate that hybrid schemes offer a scalable solution for securing communications in the post-quantum era.

Resum

Aquesta tesi investiga la transició cap a la resiliència criptogràfica contra la computació quàntica. Els sistemes criptogràfics de clau pública actuals, incloent-hi RSA i ECC, la seguretat dels quals es basa en la intractabilitat de la factorització d'enters i el problema del logaritme discret (DLP), es veuen compromesos per l'algorisme de Shor. Les primitives de clau simètrica també es veuen afectades per la computació quàntica a través de l'algorisme de Grover, que obliga a augmentar la longitud dels paràmetres de seguretat, ja que en redueix el nivell de seguretat.

Per mitigar aquesta vulnerabilitat, aquest treball analitza primitives de criptografia postquàntica (PQC) basades en reticles, que es fonamenten en la dificultat dels problemes *Learning With Errors* (LWE) i *Short Integer Solution* (SIS), centrant-se específicament en les implementacions estandarditzades com ara ML-KEM i ML-DSA.

Finalment, es dissenya un protocol de handshake autenticat híbrid (Hybrid Authenticated Handshake Protocol), que integra tant X25519 com ML-KEM mitjançant HKDF, alhora que incorpora nonces, signatures compostes i HMAC per a la integritat. Les proves de rendiment demostren que els esquemes híbrids ofereixen una solució escalable per a protegir les comunicacions en l'era postquàntica.

Resumen

Esta tesis investiga la transición hacia la resiliencia criptográfica frente a la computación cuántica. Los actuales sistemas criptográficos de clave pública, incluidos RSA y ECC, cuya seguridad se basa en la intratabilidad de la factorización de enteros y en el problema del logaritmo discreto (DLP), se ven comprometidos por el algoritmo de Shor. Las primitivas de clave simétrica también se ven afectadas por la computación cuántica a través del algoritmo de Grover, lo que obliga a aumentar la longitud de los parámetros de seguridad, ya que reduce su nivel de seguridad.

Para mitigar esta vulnerabilidad, este trabajo analiza primitivas de criptografía poscuántica (PQC) basadas en retículos, que se fundamentan en la dureza de los problemas *Learning With Errors* (LWE) y *Short Integer Solution* (SIS), centrándose específicamente en implementaciones estandarizadas como ML-KEM y ML-DSA.

Por último, se diseña un protocolo de *handshake* autenticado híbrido (*Hybrid Authenticated Handshake Protocol*), que integra tanto X25519 como ML-KEM a través de HKDF, al tiempo que incorpora *nonces*, firmas compuestas y HMAC para garantizar la integridad. Las pruebas de rendimiento demuestran que los esquemas híbridos ofrecen una solución escalable para proteger las comunicaciones en la era poscuántica.

Table of Contents

Chapter 1	1
Introduction to Cryptography and Quantum Vulnerabilities	1
1.1 Algebraic Background: Groups and Fields	1
1.1.1 Groups and Basic Definitions: Abelian and Cyclic Structures.....	1
1.1.2 Finite Groups: Subgroups, Order, and Lagrange's Theorem	1
1.1.3 Finite Fields: Arithmetic and Multiplicative Inverses	2
1.1.4 The Discrete Logarithm Problem (DLP) as a Security Primitive	2
1.2 Vulnerabilities of Current Public-Key Cryptography	3
1.2.1 The Integer Factorization Problem (RSA)	3
1.2.1.1 Security Reduction to the Integer Factorization Problem	3
1.2.2 Discrete Logarithm Problem (DLP) in Finite Cyclic Groups.....	3
1.2.2.1 Diffie-Hellman Key Exchange.....	4
1.2.2.2. ElGamal Encryption	4
1.2.3 Elliptic Curve Cryptography (ECC)	4
1.2.3.1 Elliptic Curve Groups over Finite Fields.....	4
1.2.3.2 The Elliptic Curve Discrete Logarithm Problem (ECDLP)	5
1.3 Shor's Algorithm	5
1.3.1 Quantum Computing Fundamentals	5
1.3.2 Overview of Shor's Algorithm	6
1.3.3 Shor's Algorithm for Integer Factorization (RSA)	6
1.3.4 How Shor solves the discrete logarithmic problem	7
1.4 Practical Implications & the Transition to Post-Quantum Cryptography.....	8
Chapter 2	9
Post-Quantum Cryptography and The Hybrid Transition.....	9
2.1 The Quantum Threat Spectrum: Shor vs. Grover.....	9
2.1.1 Shor's Algorithm and the collapse of Asymmetric Primitives.....	9
2.1.2 Grover's Algorithm and Symmetric Cryptography:	9
2.1.3 The NIST Standardization Process:	10
2.2 Mathematical Foundations of Lattice-Based Cryptography	10
2.2.1 Introduction to Lattices:	10
2.2.1.1 Good Basis vs. Bad Basis	11
2.2.2 Hard Problems (SVP & LWE):	11
2.2.2.1 Shortest Vector Problem (SVP)	11
2.2.2.2 Learning With Errors (LWE).....	11

2.2.2.3 Short Integer Solution (SIS/M-SIS)	12
2.2.3 Structured LWE Variants	12
2.2.3.1 Ring-LWE.....	12
2.2.3.2 Ring-LWE Problem.....	12
2.2.3.3 Module-LWE	13
2.2.4 Quantum Resistance of Lattices	13
2.3 Post-Quantum Key Exchange.....	13
2.3.1 KEM Paradigm vs Classical Key Exchange.....	13
2.3.1.1 The Theoretical Shift	14
2.3.1.2 Practical Consequence for Software Design.....	14
2.3.2 ML-KEM (CRYSTALS-Kyber, FIPS 203)	15
2.3.3 Security and Implementation Considerations	16
2.4 Post-Quantum Digital Signatures	16
2.4.1 Role of Digital Signatures in Secure Protocols	17
2.4.2 NIST-Selected Signature Schemes	17
2.4.2.1 CRYSTALS-Dilithium	17
2.4.2.1.1 Mathematical Structure and the "Abort" Mechanism.....	17
2.4.2.1.2 Algorithmic Workflow	18
2.4.2.2 Falcon	19
2.4.2.2.1 NTRU Lattices Structure	19
2.4.2.2.2 Gaussian Sampling in Falcon.....	20
2.4.2.2.3 Algorithmic Workflow	20
2.4.3 Comparative Analysis of PQ Signature Schemes	21
2.5 The Mitigation Strategy: Hybrid Cryptography	22
2.5.1 The "Harvest Now, Decrypt Later" Threat:.....	22
2.5.2 The Hybrid Paradigm:	22
2.5.3 Cryptographic Combiners and Deployment Asymmetry.....	22
Chapter 3	23
Practical Part:.....	23
3.1 Proposed Architecture: The Hybrid Paradigm	23
3.1.1 Defense-In-Depth Principle	23
3.2 Primitives (Building Blocks)	23
3.2.1 HMAC & SHA-256	24
3.2.1.1 Cryptographic Role and Vulnerability Mitigation	24
3.2.1.2 Algorithmic Structure and Workflow.....	24
3.2.1.3 Practical Implementation Analysis.....	25

3.2.2 HMAC-based Key Derivation Function (HKDF)	25
3.2.2.1 Design.....	25
3.2.2.2 Implementation & Analysis	25
3.2.3 X25519 (Classical Key Exchange)	26
3.2.3.1 KEM Design	26
3.2.3.2 Implementation & Analysis	26
3.2.4 Kyber/ML-KEM (Post-Quantum Key Exchange)	26
3.2.4.1 Design.....	27
3.2.4.2 Implementation & Analysis	27
3.2.5 ECDSA/secp256r1 (Classical Digital Signature)	27
3.2.5.1 Design.....	27
3.2.5.2 Implementation & Analysis	27
3.2.6 Dilithium/ML-DSA (Post-Quantum Digital Signature).....	28
3.2.6.1 Design.....	28
3.2.6.2 Implementation & Analysis	28
3.3 Protocol Construction	29
3.3.1 Hybrid Key Exchange Construction	29
3.3.1.1 Design.....	29
3.3.1.2 Implementation & Analysis	29
3.3.2 Hybrid Authentication Construction	30
3.3.2.1 Design.....	30
3.3.2.2 Implementation & Analysis	31
3.3.3 Authenticated Hybrid Handshake Protocol	31
3.3.3.1 Protocol Design & Flow	31
3.3.3.2 Security Analysis.....	32
3.4 Benchmark and Measurements.....	32
3.4.1. Analysis of Results.....	33
3.4.2 Synthesis and Technical Interpretation	35
3.4.3 Practical Recommendations for Deployment	36
Conclusions & Future Work.....	37

List of Figures

Figure 1: Network payload and IP fragmentation across the four evaluated handshake scenarios.	33
Figure 2: Total computational latency and statistical dispersion (σ) across the four evaluated handshake scenarios.	34
Figure 3: Functional timing breakdown of individual handshake phases across the evaluation profiles.	35
Figure 4: Terminal output demonstrating the successful execution and session key matching of the Hybrid Handshake verification test.	46
Figure 5: Terminal output showing the key pair generation, valid signature verification, and error-handling tests for classical ECDSA (P-256) signatures.	47
Figure 6: Terminal output showing the key pair generation, public/private key size details (in bytes), valid signature verification, and error-handling tests for Post-Quantum Dilithium (ML-DSA-44) signatures.	47
Figure 7: Terminal output of the Dual Signature Hybrid Authentication test suite, showing successful verification of classical and post-quantum signatures, along with the effective rejection of corrupted signature attacks.	48
Figure 8: Terminal execution trace of the complete hybrid authenticated handshake protocol, showing the sequential state transitions from initialization to successful shared secret derivation.	49
Figure 9: Terminal output demonstrating post-handshake HMAC integrity validation and a simulated Man-in-the-Middle (MITM) attack detection resulting in an automated handshake abortion.	49

List of Tables

Table 1: Architectural and Structural Comparison of Post-Quantum Signature Primitives.....	21
Table 2: Performance and Cryptographic Synthesis of the Evaluated Handshake Profiles.	35

Chapter 1

Introduction to Cryptography and Quantum Vulnerabilities

For decades, Classical cryptography has provided the mathematical foundations required to guarantee confidentiality, integrity and authenticity in digital communications. However, the rise of quantum computing is redefining the boundaries of computational hardness. Consequently, many algorithmic primitives considered to be computationally infeasible are now facing an inevitable deadline (Mosca, 2018). To address this transition, a rigorous analysis of the cryptography's core components must be performed to identify their vulnerabilities.

From a structural viewpoint, modern digital security is based on three main pillars: symmetric-key algorithms, asymmetric-key (public-key) cryptography, and cryptographic protocols. Although cryptographic hash functions are considered as a fundamental primitive in this field, they are not treated separately in this chapter, as they share many properties with symmetric constructions (EITCA Academy, 2025).

While symmetric schemes are computationally efficient, they face a fundamental challenge: the key distribution problem, which requires the key to be shared safely before the communication begins. Public-key cryptography (Diffie & Hellman, 1976) was specifically created to address this conflict utilizing linked asymmetric key pairs: an openly distributed public key and a confidential private key. Modern cryptographic protocols such as Virtual Private Networks (VPNs) and Transport Layer Security (TLS) integrate these public-key primitives to provide computationally secure key encapsulation mechanisms (KEMs) and digital signatures.

1.1 Algebraic Background: Groups and Fields

Before identifying and discussing the vulnerabilities of these cryptographic architectures, it is essential to establish the algebraic foundations on which they are built. This section introduces some important fundamentals and principles of abstract algebra such as groups, subgroups, finite groups and cyclic groups. All these algebraic concepts are crucial to completely understand the workflow of public-key algorithms whose computational security is based on the intractability of mathematical hard problems, such as integer factorization and the Discrete Logarithm Problem (DLP) (Odlyzko, 2000).

1.1.1 Groups and Basic Definitions: Abelian and Cyclic Structures

A group $(G, \circ)$ consists of a set G together with a binary operation $\circ$ defined on G (e.g., addition or multiplication) satisfying the following properties:

1. Closure: $\forall a, b \in G$, it holds that $a \circ b \in G$.
2. Associativity: $\forall a, b, c \in G$ it holds that $a \circ (b \circ c) = (a \circ b) \circ c$.
3. Identity Element: $\exists e \in G$ such that $\forall a \in G, a \circ e = e \circ a = a$.
4. Inverse: $\forall a \in G, \exists a^{-1} \in G$ such that $a \circ a^{-1} = a^{-1} \circ a = e$.

If the operation is also commutative, i.e., $\forall a, b \in G, a \circ b = b \circ a$, then G is called an Abelian group. Modern public-key cryptography relies on cyclic groups. A group G is cyclic if there exists an element $g \in G$, called the generator, such that every element in G can be expressed as a power of g :

$$\forall a \in G, \exists n \in \mathbb{Z} \text{ such that } a = g^n$$

1.1.2 Finite Groups: Subgroups, Order, and Lagrange's Theorem

While public-key algorithms rely on cyclic properties, practical implementations require dealing with finite groups, which are defined as follows: A group $(G, \circ)$ is said to be finite if it contains a finite number

of elements, denoted as the order or cardinality ($|G|$). Within these, subgroups are found: a subset $H \subseteq G$ that is itself a group under the same operation.

Lagrange's Theorem connects a group and its subgroups. It states that, for every subgroup $(H, \circ)$ of a finite group $(G, \circ)$ it holds that $\text{ord}(H)$ must divide $\text{ord}(G)$: $|H| \mid |G|$. Similarly, the order of any element $a \in G$, (the smallest positive k such that $a^k = e$) must also divide $|G|$. This integer k denotes exactly the size of the subgroup generated by a and, because of the Lagrange's Theorem, the order of any element must divide the order of the group.

1.1.3 Finite Fields: Arithmetic and Multiplicative Inverses

Advanced cryptographic constructions, notably Elliptic Curve Cryptography (ECC), operate over more complex algebraic structures known as finite fields. A set F with two binary operations, addition (+) and multiplication ($\cdot$) constitutes a field if satisfies the following criteria:

1. The set is an Abelian group under addition, to ensure that every element $a \in F$ has an additive inverse $-a$, which allows the subtraction operation. This is needed, for example, in Elliptic Curve Cryptography to define the negative of a point on the curve.
2. Non-zero elements must be an Abelian group under multiplication: This guarantees that for every element $a \in F$ exists a multiplicative inverse a^{-1} such that:

$$a \cdot a^{-1} \equiv 1$$

In this project, the field F will be a prime field—a field with p elements, where p is a prime. Throughout this project, this field will be denoted as Z_p and all operations over it are performed $\text{mod } p$. The existence of multiplicative inverses for all non-zero elements is a fundamental requirement for elliptic curve arithmetic (Hankerson et al., 2004). Point addition is defined through the computation of the slope λ :

$$\lambda = (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \text{ mod } p$$

This operation forms the basis of scalar multiplication ($Q = d \cdot G$), which constitutes the fundamental operation underlying X25519 key exchange.

1.1.4 The Discrete Logarithm Problem (DLP) as a Security Primitive

The security of modern asymmetric cryptography is based on One-Way Functions: operations easy to compute but computationally infeasible to reverse. A primary example is exponentiation in a finite cyclic group. Given a group G of order n , a generator g , and an element $h \in G$, the DLP consists of finding the integer x where $0 \leq x \leq n - 1$ such that: $g^x = h$.

While h can be computed from g and x in polynomial time, no classical algorithms can find x in polynomial time for sufficiently large groups. These two implementations are the most relevant in modern cryptography:

1. The Multiplicative Group $(Z_p^*, \cdot)$: Used in the original Diffie-Hellman and ElGamal protocols. Its security is based on the hardness of finding x in the congruence $g^x \equiv h \pmod{p}$.
2. Elliptic Curve Groups $(E(Z_p), +)$: Elements are the points (x, y) satisfying the Weierstrass equation:

$$y^2 = x^3 + Ax + B \text{ mod } p$$

where $x, y \in Z_p$.

The set also includes the point at infinity (O) known as the identity element. The group operation is point addition (+). The problem (ECDLP) involves, given a generator P , finding the integer d (secret key) such that $dP = P + \dots + P$ (d times) $= T$, where P is the generator, T is the resulting public point, $1 \leq d \leq \#E$ and $\#E$ is the number of points in the curve (Proos & Zalka, 2003).

While both $(Z_p^*, \cdot)$ and $(E(Z_p), +)$ are cyclic groups where the DLP can be defined, the second one offers higher resistance to classical cryptanalysis. More concretely, for $(Z_p^*, \cdot)$, there exist algorithms

such as the General Number Field Sieve (GNFS) that can solve the problem in sub-exponential time. In contrast, for well-chosen elliptic curves, only exponential-time algorithms are known, which allows the use of significantly smaller keys to achieve the same level of security. This efficiency is the primary reason for the adoption of standards like X25519 in contemporary secure communications.

1.2 Vulnerabilities of Current Public-Key Cryptography

The security of asymmetric algorithms is based on the difficulty of solving specific mathematical problems within a reasonable computation time. As defined in standard cryptographic literature (Paar & Pelzl, 2010), three main Public-Key Algorithm Families are of practical relevance in the classical computing era:

- Integer-Factorization Schemes: Because it is difficult to factor large integers. RSA is the most prominent representative.
- Discrete Logarithm Schemes: Reliant on the discrete logarithm problem in finite groups, including Diffie-Hellman (DH) key exchange, ElGamal encryption, and the Digital Signature Algorithm (DSA).
- Elliptic Curve (EC) Schemes: A generalization of the Discrete Logarithm Problem applied to the algebraic structure of elliptic curves, including Elliptic Curve Diffie-Hellman (ECDH) and the Elliptic Curve Digital Signature Algorithm (ECDSA).

Currently, no effective attacks against any of these schemes exist using classical computers, provided that the parameters as key sizes are chosen according to modern standards (e.g., RSA-3072 or ECC-256). However, it is critical to note that should large-scale quantum computers become available, none of these three established families will remain secure, as they all rely on number-theoretic functions that quantum algorithms can solve in polynomial time (Bernstein et al., 2009).

1.2.1 The Integer Factorization Problem (RSA)

RSA is an asymmetric encryption scheme whose security is based on the difficulty of factoring a large composite modulus N into its constituent primes (Rivest et al., 1978). This modulus is generated by choosing two uniformly random prime numbers p and q and computing their product $N = p \cdot q$.

This modulus is essential for both public and private keys. The detailed mathematical workflow, including both the calculation of Euler's totient function $\phi(N)$ and the private exponent d , is provided in Appendix A.1.

1.2.1.1 Security Reduction to the Integer Factorization Problem

The security of RSA is fundamentally linked to the Integer Factorization problem. Within a cryptographic context, an adversary cannot recover the secret key d unless it factors N . As finding the secret key is equivalent to factoring, it is computationally infeasible to recover p and q in polynomial time for classical computers.

Although classical integer factorization algorithms exist—such as Fermat's factorization method or the General Number Field Sieve (GNFS), which is currently the most efficient classical approach—they operate in sub-exponential time. Consequently, for standardized RSA key lengths, the computational complexity of classical factorization remains infeasible, thereby ensuring the scheme's security.

1.2.2 Discrete Logarithm Problem (DLP) in Finite Cyclic Groups

As stated in Section 1.1.4, the DLP is the “One-Way Function” used in several categories of asymmetric algorithms. The security of the following defined protocols is based on the general algebraic properties of a finite cyclic group $(G,*)$ of order n . Specifically, if an adversary can solve the DLP in G (recovering a given g^a), the protocol collapses. Conversely, as long as the DLP is computationally infeasible (hard), the protocol remains secure. This logic is similarly applied across different algebraic structures, whether

operating in classical multiplicative groups or additive subgroups utilized in Elliptic Curve Cryptography (ECDLP).

1.2.2.1 Diffie-Hellman Key Exchange

The Diffie-Hellman (DH) protocol is the best-known exchange mechanism based on the hardness of the DLP (Diffie & Hellman, 1976). It is important to highlight that DH is not an encryption scheme, but a key exchange protocol that allows two parties to establish a shared secret over an insecure channel. This shared secret is particularly useful for symmetric cryptography, since this secret can be used as a key for symmetric algorithms such as AES (EITCA Academy, 2025).

As stated before, the security of this protocol is ensured because it is considered computationally difficult for classical computers to solve the discrete logarithm problem. The detailed workflow of this exchange protocol is provided in Appendix A.2.

1.2.2.2. ElGamal Encryption

ElGamal is an asymmetric encryption algorithm that, as RSA does, uses modular arithmetic on large numbers. It builds on the principles of the DH key exchange, and its security is reduced to the difficulty of solving the DLP in the group G considered. As commented before, the existence of a multiplicative inverse in the field (or the group structure) is what allows for the decryption process, as the blinded value used during encryption is cancelled. Again, its detailed three-staged workflow (key generation, encryption, decryption) is detailed in the Appendix A.3.

1.2.3 Elliptic Curve Cryptography (ECC)

As stated in standard cryptographic literature (Paar & Pelzl, 2010), Elliptic Curve Cryptography (ECC) forms, together with RSA and discrete logarithm schemes, the third family of most widely used public-key algorithms. ECC is based on the generalized DLP, which means that protocols like the Diffie-Hellman key exchange can be implemented using elliptic curves. The primary advantage of ECC is its efficiency: it provides the same level of security as RSA or classical DL-systems with considerably shorter key lengths and ciphertexts. For example, a 256-bit ECC key offers a security level comparable to a 3072-bit RSA key. This reduction in key size leads to faster computations and lower bandwidth requirements.

1.2.3.1 Elliptic Curve Groups over Finite Fields

To understand the workflow of ECC functions, it is necessary to define its underlying mathematical structure. An elliptic curve over a finite field Z_p is defined by the Weierstrass equation:

$$y^2 = x^3 + Ax + B \bmod p$$

where $A, B \in Z_p$ are constants that must satisfy the condition known as non-singularity:

$$(4A^3 + 27B^2) \not\equiv 0 \bmod p$$

This ensures that the curve has neither self-intersections nor vertices, which is essential for cryptographic security (Hankerson et al., 2004). To achieve a group structure $(E, +)$ a binary operation must be defined, known as Point Addition in ECC. As established in standard literature (Koblitz, 1987), the set of all pairs (x, y) that satisfy the equation, together with the point at infinity (known as O) form an abelian group under these rules:

- **Neutral Element:** The point at infinity (O) is the neutral element of the group. For any point P on the curve, it holds that $P + O = P$.
- **Point Addition ($P + Q$):** Given two points $P = (x_1, x_2)$ and $Q = (q_1, q_2)$ on the curve (where $P \neq Q$), their sum is defined by drawing a line through them, resulting in a third point of intersection, which is mirrored in the x-axis, known as $R = P + Q$. The resulting coordinates are computed by performing modular arithmetic operations. If $P = Q$ the operation is known as Point Doubling and can be written as $P + P = 2P = R$.

- **Scalar Multiplication:** A cyclic group is generated by adding d times a base point P to itself ($P + \dots + P = dP = Q$). This is known as scalar multiplication, and it is the main operation of ECC.

The security and functionality of ECC depends on the algebraic structure of the elliptic curve group. The detailed procedural workflow for key exchange and domain parameter selection is provided in Appendix A.4.

1.2.3.2 The Elliptic Curve Discrete Logarithm Problem (ECDLP)

The security of Elliptic Curve Cryptography is fundamentally based on the difficulty of the ECDLP. The definition of this problem is: Given an elliptic curve E defined over a finite field Z_p , a publicly known base point G , and a public key $Q = dG$, the problem is stated as follows: Given G and Q , determine the private key d , which satisfies $1 \leq d \leq \#E$ (where $\#E$ represents the total number of points on the curve). While computing $Q = dG$ is efficient, the reverse problem (determining d from G and Q) is computationally infeasible for appropriately chosen curve parameters, since no known classical algorithm can solve it in sub-exponential time. Also, it is important to note that there are not any kind of algebraic structures that allow any kind of mathematical attacks. Consequently, elliptic-curve-based systems can achieve strong security with substantially smaller key sizes than RSA or traditional discrete logarithm schemes (Menezes et al., 1996).

The ECDLP forms the security foundation of several important elliptic-curve-based protocols:

- **Elliptic Curve Diffie-Hellman (ECDH):** As mentioned before, in this protocol the secret is the point $K = abG$, which is obtained by both parties computing $K = aB$ or $K = bA$. The only way that it is known in which an adversary can compute the shared key requires to compute $a = \log_G A$ or $b = \log_G B$, which is an instance of the ECDLP.
- **Digital Signatures:** Forging a valid signature or recovering the signing key requires computing a secret scalar from public point multiplications. The recovery of the nonce or the private key fundamentally reduces to solving an ECDLP on the underlying group. Being able to derive this scalar would allow an attacker to break the semantic security of the system, and this again reduces to solving an ECDLP.

In any of these cases, compromising the system requires recovering a secret scalar from a publicly known point multiplication—a task that reduces to solving the ECDLP, which remains computationally infeasible for classical computers.

1.3 Shor's Algorithm

The importance of Shor's algorithm lies in its ability to factor large integers in polynomial time and solve the discrete logarithm problem in a quantum computer, thus making these classical cryptographic schemes obsolete (Shor, 1994).

1.3.1 Quantum Computing Fundamentals

To understand how Shor's algorithm works, it is necessary to learn some properties of quantum computers and some basic differences between them and classical ones. While in classical computers bits are used as the unit of information (which can have just one of two potential values: 0 or 1), quantum computers use qubits, which can exist in a superposition of states. The state of a qubit is represented as:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

where α and β are both complex numbers that represent probability amplitudes. It must be satisfied that:

$$|\alpha|^2 + |\beta|^2 = 1$$

This property enables quantum computers to investigate numerous choices or solutions concurrently (Nielsen & Chuang, 2010). Two other concepts are crucial for this quantum speedup:

- Entanglement: This property establishes a correlation between the states of two qubits. Once they are entangled, their states depend on each other; this allows quantum computers to solve complex problems more efficiently than classical computers (Preskill, 2018).
- Interference: This property is the key to quantum algorithms like Shor. It amplifies the probability of measuring the correct solution while cancelling out the probability of getting a wrong one, which is not possible in classical computers (Arute et al., 2019).

The practical application of these properties relies on the concept of the oracle. Basically, a quantum oracle is a Blackbox function that marks the solution or the right answer from a lot of possibilities without revealing it. By combining superposition and interference the correct answer is amplified, enabling quantum algorithms to solve problems asymptotically faster than the classical ones.

It is crucial to note that this superposition remains until the qubit is observed. In quantum mechanics, this act of observation is known as measurement, which causes the collapse of the wavefunction. At that moment, the qubit is forced to choose one of the two definite classical outcomes based on its probability amplitudes. This distinction is what allows quantum computers to work with a vast space of possibilities before a result is measured.

1.3.2 Overview of Shor's Algorithm

Fundamentally, Shor's algorithm reduces both the integer factorization and the discrete logarithm problems to the period-finding problem. Instead of attacking the cryptographic functions directly through brute force, the algorithm constructs a periodic function $f(x)$ and utilizes quantum superposition, interference, and the Quantum Fourier Transform (QFT) to extract its hidden period r efficiently. The conceptual architecture is identical for breaking both RSA and ECC, although the specific algebraic instantiation of the function $f(x)$ differs depending on whether the target is factorization or the DLP.

1.3.3 Shor's Algorithm for Integer Factorization (RSA)

This section explores the steps followed by Shor, succeeded by a couple of practical examples. In this case, the goal is to find the components p and q that are used to compute N .

The process begins by choosing a random integer a such that $1 < a < N$ and it must be verified that a and N are coprimes, which means that:

$$\gcd(a, N) = 1$$

If they are not coprime, the process must stop there because a factor has been found. If they are coprime, the quantum phase of the algorithm begins. The purpose is to find the period of the function:

$$f(x) = a^x \bmod N$$

This is defined as the smallest integer r such that:

$$a^r \equiv 1 \bmod N$$

Once r has been computed (and assuming r is even), the mathematical core of the algorithm lies in algebraic factorization. The congruence can be rewritten as $a^r - 1 \equiv 0 \bmod N$. Applying the difference of squares property, the expression expands to:

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \bmod N$$

This implies that the product of the two terms is a multiple of N . The term $a^{r/2} - 1$ cannot be congruent to zero modulo N , since it would contradict the definition of r as the smallest positive integer such that $a^r \equiv 1 \bmod N$. Consequently, two different cases must be considered:

- $a^{r/2} + 1 \equiv 0 \bmod N$, which implies that $a^{r/2} \equiv -1 \bmod N$. At this point, the current value of a is discarded and the algorithm is restarted with a new random choice, since the factorization only yields trivial factors and no information about the prime factors of N can be extracted.

- b) $a^{r/2} + 1 \not\equiv 0 \pmod{N}$, and hence neither $a^{r/2} - 1$ nor $a^{r/2} + 1$ is divisible by N (while their product is), both terms are non-trivial factors of N . These factors can be recovered with the Euclidean Algorithm:

$$p = \gcd(a^{r/2} - 1, N), q = \gcd(a^{r/2} + 1, N)$$

The process of finding r is computationally unfeasible for a classical computer, this is where quantum computers become important. By using superposition (the property explained before), $f(x)$ can be computed for many different values of x in parallel, instead of computing each x value one by one. This is achieved through these steps:

1. Quantum Parallelism: Evaluating $f(x)$ simultaneously using two registers: one contains all possible values of x , and the other one the corresponding values of $f(x)$.
2. Quantum Fourier Transform (QFT): Because the quantum architecture encodes the ensemble of possible inputs simultaneously within a single quantum state, the QFT transforms the quantum state from time domain to frequency domain, which in this case is very useful for identifying the period of the function $f(x)$, enabling the extraction of the repeating pattern.
3. Interference and Measurement: Probability amplitudes of different states combine via quantum interference; incorrect results cancel each other out, while the correct periodic ones are amplified (Shor, 1994). This creates distinct frequency peaks that are multiples of $1/r$.

Finally, by measuring the quantum state, a single outcome is obtained containing information about the period. The next step is to process this measurement classically—using techniques such as continued fraction expansion—to extract the exact value of r , which allows to find the non-trivial factors of N (Arute et al., 2019; Preskill, 2018).

1.3.4 How Shor solves the discrete logarithmic problem

As indicated above, the security of Diffie-Hellman, ElGamal and ECC is based on the hardness of solving the discrete logarithm problem (DLP). Shor's algorithm can also solve this problem efficiently on a quantum computer by transforming the DLP into a period-finding problem, as in the factorization case (Shor, 1997; Proos & Zalka, 2003).

In a cyclic group G of order r with a generator g and an element $h = g^x$, the goal is to find the exponent x . Classically, this problem is extremely hard for large groups, as no known algorithms can solve it in polynomial time. However, the DLP can be expressed in different algebraic structures, but Shor's approach remains the same: defining a function $f(a, b)$ whose computation depends on the specific algebraic structure:

- In multiplicative groups (Z_p^*): In this case modular exponentiation is used:

$$f(a, b) = g^a h^{-b} \pmod{p}$$

- In Elliptic Curve Groups ($E(F_p)$): Scalar multiplication is used:

$$f(a, b) = aG - bH$$

The function is periodic in two dimensions: its values repeat ($f(a, b) = f(a', b')$) whenever the pairs (a, b) and (a', b') satisfy:

$$a - a' \equiv x(b - b') \pmod{r}$$

The quantum computer then creates a superposition of all possible pairs (a, b) , evaluating the function $f(a, b)$ in parallel. After this, it performs a two-dimensional Quantum Fourier Transform (QFT), which reveals the periodic structure of f .

When the state is measured, the resulting data encodes linear relations between a, b and x . With several measurements, it is possible to determine x by solving a small system of linear equations. In summary, Shor's algorithm for DLP utilizes the same principles of quantum parallelism, interference, and Fourier analysis to find hidden periods in a two-dimensional version of the problem. Once x is found, the discrete logarithm problem is effectively solved, and therefore, the security of protocols like Diffie-Hellman, ElGamal, and ECC is broken in polynomial time (Proos & Zalka, 2003).

1.4 Practical Implications & the Transition to Post-Quantum Cryptography

Shor's algorithm demonstrates that all cryptographic systems based on integer factorization (RSA) and the discrete logarithmic problem (Diffie-Hellman, ElGamal, and ECC) are expected to become vulnerable once large-scale reliable quantum computers become available (Bernstein et al., 2009; NIST, 2024). Currently, these primitives are deployed across modern digital infrastructure, including HTTPS, VPNs, secure emails and government communications networks; therefore, a working quantum computer would impact a great deal of existing digital security. While the best-known algorithms classically would require an unreasonably long period of time to break a 2048-bit RSA key, estimates suggest that a fault-tolerant quantum computer utilizing approximately 4000 logical qubits could accomplish it in less than three hours (NIST, 2024).

Although there are no quantum computers at present capable of implementing Shor's algorithm at a relevant scale, there have been significant advancements. A breakthrough was achieved in 2019, when Google demonstrated "Quantum Supremacy" using its 53-qubit Sycamore processor by completing a specific mathematical task in approximately 200 seconds that would take classical supercomputers thousands of years to complete (Arute et al., 2019). This accomplishment presents a tangible risk known as "Harvest Now, Decrypt Later" (HNDL). An attacker can collect encrypted data today and store it until quantum computers are powerful enough to recover the keys from said data and then read all of it. Consequently, organizations have already started to plan ways to substitute their current public-key algorithms for new schemes that are considered secure even against quantum computers (Bernstein et al., 2009; NIST, 2024).

Because of this threat, the need for new cryptographic standards is urgent. Post-quantum cryptography (PQC) studies classical algorithms designed to resist both classical and quantum attacks, by avoiding factorization and discrete logarithm problems, utilizing other mathematical problems for which no efficient quantum attacks are known (Bernstein et al., 2009; NIST, 2024). Important families include lattice-based cryptography (e.g., CRYSTALS-Kyber and Dilithium), code-based cryptography (such as Classic McEliece), hash-based signature schemes (such as SPHINCS+), and multivariate quadratic equations. To guide this transition, the U.S. National Institute of Standards and Technology (NIST) launched a project to select and standardize post-quantum algorithms. Following the selection of the first candidates, NIST officially released the finalized Federal Information Processing Standards (FIPS 203 for ML-KEM, FIPS 204 for ML-DSA, and FIPS 205 for SLH-DSA) in August 2024, driving the global rollout and widespread implementation of these primitives across infrastructure over the course of this decade (NIST, 2024). In practice, this means that Shor's algorithm has not only revealed the limits of today's public-key cryptography but has also forced the community to design a new generation of cryptographic algorithms. These post-quantum schemes, and how they aim to replace current systems, will be analysed in detail in the next chapter.

Chapter 2

Post-Quantum Cryptography and The Hybrid Transition

Chapter 1 highlighted the vulnerabilities of current public-key cryptography against quantum computing architectures. This chapter focuses on the possible solutions to address these vulnerabilities by analysing the new standards and schemes emerging from NIST Post-Quantum (PQC) standardization process, considering the essential mathematical foundations of Lattice-based cryptography, and examining in depth the hybrid architectures, crucial for facing the “Harvest Now, Decrypt Later” (HNDL) threat during this transitional period.

2.1 The Quantum Threat Spectrum: Shor vs. Grover

Before diving into post-quantum primitives, a clear line must be drawn between the two main quantum algorithms which affect modern cryptography. While Shor’s algorithm fundamentally invalidates the mathematical assumptions underlying classical asymmetric cryptosystems, Grover’s algorithm alters the security parameters of symmetric encryption by providing a quadratic speedup for brute-force search.

2.1.1 Shor’s Algorithm and the collapse of Asymmetric Primitives

As referenced in Section 1.3, Shor’s algorithm represents a structural threat to current public-key infrastructure due to its capacity to efficiently solve the two core mathematical challenges that protect modern networks:

- The Integer Factorization Problem, which guarantees the security of RSA architectures (Shor, 1994).
- The Discrete Logarithm Problem (DLP), including its elliptic curve variant (ECDLP), which forms the basis of both Finite Field Diffie-Hellman and Elliptic Curve Cryptography (ECC) (Shor, 1994; National Academies of Sciences, Engineering, and Medicine [NASEM], 2019).

Shor’s period-finding algorithm operates in polynomial time on quantum computers, lowering the execution time compared to classical sub-exponential algorithms. Regarding RSA, the algorithm can find the period of a modular function by using Quantum Fourier Transform, exposing the primer factors of the integer $N = p \cdot q$. Similarly, it breaks those systems based on the DLP by resolving the periodic structure of the underlying algebraic groups, allowing an attacker to recover the private exponent efficiently (NASEM, 2019).

Consequently, new mathematical problems immune to the speed-up provided by the quantum computing arrival are needed, since all public-key protocols which are currently used (VPNs, TLS) will be vulnerable to quantum adversaries (Mavroeidis et al., 2018).

2.1.2 Grover’s Algorithm and Symmetric Cryptography:

While Shor’s algorithm compromises asymmetric cryptography, it does not compromise symmetric schemes as well. Instead, symmetric ciphers and cryptographic hash functions are affected by Grover’s quantum search algorithm (Grover, 1996).

As established in previous sections, brute-force represents the best-known attack against symmetric encryption schemes. Assuming an adversary possesses a plaintext/ciphertext pair, the execution of this attack requires encrypting the plaintext with all possible keys and then searching for the known ciphertext. If this attack is executed against AES-128, it requires 2^{128} steps. The same attack utilizing Grover’s algorithm will require 2^{64} operations, which is a security level of 64, considered inadequate for long-term protection. This problem is solved by just increasing the key length. This is why AES-256 was designed, because although it is affected by Grover’s algorithm and its security level is reduced

from 256 to 128, it is currently considered as acceptable against quantum and classical attacks (NIST, 2016).

Hash functions are similarly affected by this phenomenon. For example, primitives within SHA-2 and SHA-3 are vulnerable to Grover's algorithm referring to preimage and second-preimage resistance. As observed with symmetric encryption, the effect is a square-root reduction in security strength, which is again solved by increasing the hash output key length to preserve equivalent post-quantum security levels.

It is important to understand that Grover's algorithm does not completely break symmetric cryptography, it creates the need for stricter parameter selection. By increasing key sizes and hash output lengths, symmetric primitives can remain secure also in the presence of quantum adversaries. This property is totally different to the effect of Shor's algorithm on public-key cryptography and explains why symmetric primitives continue to be used as foundational components in post-quantum systems, including key derivation functions, authenticated encryption, and hybrid key exchange architectures (Bindel et al., 2019).

2.1.3 The NIST Standardization Process:

In response to the threat caused by large-scale quantum computers, the NIST launched an international standardization process with the main purpose of identifying cryptographic algorithms capable of resisting attacks from quantum computers without weakening classical security. This effort introduced a clear separation between cryptographic primitives, distinguishing two main categories:

- Key Encapsulation Mechanisms (KEMs): Designed for secure key establishment over insecure channels (NIST, 2024).
- Digital Signature Schemes: Targeted at authentication, data integrity, and non-repudiation.

This shows an architectural shift in modern cryptographic protocols. Historically, public-key encryption schemes such as RSA were used for both encryption and key exchange. However, modern secure communication protocols no longer rely on direct public-key encryption to protect data flows (Rescorla, 2018). Instead, asymmetric primitives are only employed to establish shared symmetric keys, which are then used for encryption in symmetric schemes.

As a result, the primary role of post-quantum public-key cryptography is not data encryption but secure key establishment. For this reason, NIST focused on Key Encapsulation Mechanisms (FIPS 203) as the direct post-quantum replacement for classical Diffie–Hellman and Elliptic Curve Diffie–Hellman (ECDH) protocols.

2.2 Mathematical Foundations of Lattice-Based Cryptography

This section introduces the mathematical foundations of the Module-Lattice-Based Key Encapsulation Mechanism, ML-KEM (Kyber), together with the corresponding digital signature schemes. Lattice-based cryptography is based on the hardness of solving geometric problems in high-dimensional structures known as lattices. Solving these types of problems is computationally infeasible for both classical and quantum algorithms, which makes Lattices-based cryptography an important piece for post-quantum cryptography (Peikert, 2016).

2.2.1 Introduction to Lattices:

A lattice L is defined as a set of all linear integer combinations of a set of independent basis vectors a_i and their respective integer coefficients x_i in a n -dimensional space. Formally, let $B = a_1, \dots, a_n \in R^m$ be a set of linearly independent vectors (where $m \geq n$). The lattice generated by this basis B is defined as (Micciancio & Goldwasser, 2002):

$$L(B) = \{x_1 \cdot a_1 + \dots + x_n \cdot a_n \mid x_i \in \mathbb{Z}\} \text{ or } L(B) = \{\sum_{i=1}^n x_i \cdot a_i \mid x_i \in \mathbb{Z}\}$$

The rank of a lattice L is denoted by its number n of vectors, while m denotes the dimensional lattice space. In practice, integer lattices are used in cryptographic schemes, where its basis vectors have its

coordinates in a finite ring Z_q (Hoffstein et al., 2008), making those schemes suitable for computation in classical computers.

2.2.1.1 Good Basis vs. Bad Basis

The security of lattice-based cryptography is based on the fact that a lattice L can be generated by an infinite number of different bases. While all these bases generate the same set of points, they significantly differ in geometric quality (Goldreich et al., 1997).

- **Good Basis:** It consists of vectors that are relatively short and nearly orthogonal. Possessing a good basis makes solving the Closest Vector Problem (CVP) or the Shortest Vector Problem (SVP) computationally easy, as one can simply round the coordinates of a target point to the nearest integer coefficients (Babai, 1986). In cryptographic schemes, this good basis is kept secret and serves as the Private Key.
- **Bad Basis:** A bad basis is composed of very long vectors that are non-orthogonal. While they generate the same lattice points, they are highly misaligned. For an observer possessing only a bad basis, finding the closest lattice point to a target (CVP) or the shortest vector (SVP) becomes an intractable problem in high dimensions (Goldreich et al., 1997). In cryptographic schemes, this basis functions as the public key.

2.2.2 Hard Problems (SVP & LWE):

As mentioned before, cryptography's computational security is based on solving computationally hard problems. In the case of lattice-based systems, its security is derived from the complexity of finding specific points in a lattice. These problems are computationally unfeasible when operations scale into hundreds or thousands of dimensions (Micciancio & Regev, 2007).

2.2.2.1 Shortest Vector Problem (SVP)

The Shortest Vector Problem (SVP) is one of the central hard problems underlying lattice-based cryptography. Its formal definition is quite simple: Given a lattice L , generated by the basis vector B (expressed as $L = \zeta(B)$), the objective is to locate the shortest non-zero vector v within that specific lattice. Formally, the goal is to find a vector $v \in L$ such that its euclidean length $\|v\|$ equals the minimum distance of the lattice, denoted as $\lambda_1(L)$ (Micciancio & Goldwasser, 2002):

$$\|v\| = \lambda_1(L) = \min_{0 \neq x \in \zeta} \|x\|$$

The SVP is believed to be computationally intractable in high dimensions, and certain approximation variants are known to be NP-hard, and in high-dimensional spaces, there is no known classical or quantum algorithm that can solve the SVP in polynomial time. That is why in modern post-quantum standards like ML-KEM (Kyber) lattices are built in hundreds of dimensions.

2.2.2.2 Learning With Errors (LWE)

This problem is quite similar to the Discrete Logarithm Problem (DLP), but in this case based on lattices. The main idea is that several basis vectors $a_1, a_2, \dots, a_n \in Z_q^m$ are multiplied with secret integer coefficients $s_1, s_2, \dots, s_n$ to reach a point t on the lattice:

$$t = s_1 a_1 + s_2 a_2 + \dots + s_n a_n$$

As mentioned before, this is quite similar to the Discrete Logarithm Problem, but in this case The adversary is given the set of basis vectors a and the end point t and his goal is to find the set of coefficients s . This can be also expressed in matrix form as:

$$A \cdot s = t$$

where A is the matrix that contains the set of basis vectors, s is the set of secret integer coefficients and t the target point on the lattice. But this problem can easily be solved by, for example, applying Gaussian elimination, since the matrix can be seen as a linear equations system.

To solve this issue, it is enough to add an error vector e , turning the equation into:

$$A \cdot s + e = t \pmod{q}$$

At this point, it is necessary to clarify that now t is no longer a point in the lattice, since e completely perturbs the result. Consequently, t is a close point to the lattice, but not inside it. This transforms the problem into a computationally hard instance, since neither Gaussian elimination nor any other mathematical technique is capable to solve it without knowing also e or s (Regev, 2005).

2.2.2.3 Short Integer Solution (SIS/M-SIS)

The Short Integer Solution problem (SIS) is a hard problem in lattice-based cryptography that underpins most of the post-quantum digital signature schemes security, including CRYSTALS-Dilithium and Falcon, which will be explained in the following sections.

Its definition is quite simple: Given a random matrix $A \in \mathbb{Z}_q^{m \times n}$, the SIS problem consists of finding a non-zero vector $z \in \mathbb{Z}^n$ with small coefficients where it is satisfied that:

$$A \cdot z \equiv 0 \pmod{q}$$

This problem is the same as finding a short vector in a lattice L defined by the matrix A , which was already commented in the previous section that is computationally infeasible when parameters are chosen correctly.

The Module-SIS (M-SIS) problem is a variant of SIS which aims to improve performance and efficiency while keeping the same hardness properties. The main difference is that instead of working with integers modulo q , the M-SIS problem is defined over polynomial rings of the form (Langlois & Stehlé, 2015):

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

which will be explained in the following section. So, in this variant the goal is that given a matrix $A \in R_q^{n \times m}$, find a short vector of polynomials $z = (z_1, \dots, z_m) \in R_q^m$ such that $A \cdot z = 0$.

This version is very useful for the digital signatures field, since it allows to reduce both the computational cost and the key size while maintaining the security level needed for real-world implementations such as ML-DSA (Lyubashevsky et al., 2013).

2.2.3 Structured LWE Variants

As explained above, Standard-LWE is defined over matrices and vectors in $\mathbb{Z}_q^n$. To significantly reduce both storage and computational costs, practical lattice-based cryptography replaces matrix arithmetic with structured algebraic objects. That is, the Learning With Errors problem is constructed over polynomial rings, referred to as the Ring Learning With Errors Problem (Ring-LWE), followed by its intermediate generalization, Module-LWE (Lyubashevsky et al., 2010).

2.2.3.1 Ring-LWE

In Ring-LWE, matrix operations are replaced by computations in a polynomial ring. The most common construction used in lattice cryptography is based on cyclotomic polynomials of the form: $X^{2^l} + 1$, where $n = 2^l$. A polynomial ring is defined as (Lyubashevsky et al., 2010):

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

This ring consists of all polynomials of degree at most $n - 1$ with coefficients in $\mathbb{Z}_q$ where all coefficient arithmetic is performed $\pmod{q}$ and polynomial multiplication is reduced $\pmod{X^n + 1}$. The form of the modulus makes the reduction process simple, since:

$$x^n \equiv -1 \pmod{X^n + 1}$$

and then it follows that:

$$x^{n+1} \equiv -x, x^{n+2} \equiv -x^2 \text{ and so on}$$

2.2.3.2 Ring-LWE Problem

Given a uniformly random polynomial $a(x) \in R_q$, a secret polynomial $s(x)$ and a small error polynomial $e(x)$, the Ring-LWE problem considers samples of the form

$$(a(x), a(x) \cdot s(x) + e(x))$$

However, Ring-LWE introduces additional algebraic structure. While the problem is believed to be secure with properly chosen parameters, the presence of structure makes security reductions less conservative than in the fully unstructured Standard-LWE setting (Peikert, 2016).

2.2.3.3 Module-LWE

To balance efficiency and conservative security assumptions, Module-LWE was introduced. It combines the matrix-based structure of Standard-LWE with the polynomial arithmetic efficiency of Ring-LWE.

Instead of working with a single polynomial (Ring-LWE) or large matrices over integers (Standard-LWE), Module-LWE operates over vectors of polynomials in R_q . Formally, the problem takes the form:

$$A \cdot s + e = t \pmod{q}$$

where the matrix A is typically a square matrix of size $k \times k$, and each entry within this matrix is a polynomial belonging to R_q . The lattice dimension is computed as $n * k$, where n is the polynomial degree and k denotes the module rank or dimension. By applying this structure, efficient arithmetic is achieved via polynomial multiplication, allowing scalable security configurations (Peikert et al., 2014).

2.2.4 Quantum Resistance of Lattices

The main reason why lattice-based cryptography is considered quantum-resistant relies on the fundamental difference between its underlying hard problems and those of classical systems like RSA or ECC.

Public-key standards are broken by Shor's algorithm, which relies on finding the period of a function within a cyclic group (such as the group of integers *mod n* or the group of points on an elliptic curve). Quantum computers are highly efficient at this task by using the Quantum Fourier Transform (QFT) to expose the periodic structure of the secret key (Shor, 1997).

In contrast, lattice-based problems like SVP and LWE do not rely on periodic structures that Shor's algorithm can exploit. Furthermore, the intentional introduction of a noise vector e in LWE-based schemes serves as a safeguard; this noise prevents the efficient application of the QFT to resolve the secret coefficients. Even with the help of Grover's algorithm, which can provide a quadratic speedup for searching, the security of lattices remains solid by simply increasing the dimension n of the lattice. Therefore, these problems remain computationally hard for both classical and quantum architectures.

2.3 Post-Quantum Key Exchange

Among the finalists of the NIST standardization process mentioned above, CRYSTALS-Kyber was selected and formally standardized as Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) under the FIPS 203 standard. This section details its workflow and deepens into the structural differences from classical protocols, which directly influence the software design implementations.

2.3.1 KEM Paradigm vs Classical Key Exchange

The transition to post-quantum cryptography does not only involve replacing the mathematical assumptions underlying classical cryptographic schemes; it also affects how shared secrets are established within communication protocols. Classical protocols have traditionally relied on Diffie-Hellman key exchange mechanisms, where both parties derive a common secret through operations in a shared algebraic structure.

In contrast, many post-quantum constructions, including ML-KEM (Kyber), lack this natural algebraic property and are formulated as Key Encapsulation Mechanisms (KEMs). In this framework, one entity encapsulates a random secret that can only be recovered by the possessor of the corresponding private

key. This influences how post-quantum primitives are integrated into existing protocols, particularly in hybrid TLS constructions where classical and post-quantum shared secrets are combined.

2.3.1.1 The Theoretical Shift

For decades, the Diffie-Hellman key exchange paradigm has been the foundational mechanism for secure communications, enabling two parties to establish a shared secret in an interactive, symmetric-like process. Both parties generate a private secret, exchange public values and then perform a commutative mathematical operation to finally arrive at the same shared secret simultaneously.

The main limitation of lattice-based schemes is that they do not support the commutative property. This limitation arises from the presence of the error vector (e) in the equation, which at the same time is essential for the cryptographic hardness of the scheme (Regev, 2005). If two parties (Alice and Bob) attempt a Diffie-Hellman-like exchange in a scheme relying on Learning With Errors (LWE), they will follow this workflow:

1. Both parties agree on a shared public matrix A .
2. Alice generates her secret vector s_A and the error vector e_A , computing her public key as:

$$P_A = A \cdot s_A + e_A$$
3. The same process as above is repeated by Bob. In this case his public key is computed as:

$$P_B = A \cdot s_B + e_B$$

Now both parties attempt to compute the shared secret by multiplying their received public key by their own private secret, as in Diffie-Hellman process:

- Alice computes: $P_B \cdot s_A = (A \cdot s_B + e_B) \cdot s_A = As_B \cdot s_A + e_B \cdot s_A$
- Bob computes: $P_A \cdot s_B = (A \cdot s_A + e_A) \cdot s_B = As_A \cdot s_B + e_A \cdot s_B$

While the core mathematical term ($A \cdot s_B \cdot s_A$) is identical for both parties, the resulting error terms differ entirely: ($e_B \cdot s_A \neq e_A \cdot s_B$). Consequently, the final derived values will not match with the lattice space, although they will be mathematically close. Hence, the subsequent operations (HMAC validation, for example) will fail completely, since they require identically bit strings.

To solve this mathematical limitation, the cryptographic community shifted entirely to the Key Encapsulation Mechanism (KEM) paradigm. Instead of a mutual mathematical derivation, the KEM process is one-directional. One party (the client) locally generates a symmetric key. The client then encapsulates (encrypts) this key using the server's public key and transmits the resulting ciphertext. The server receives the ciphertext and uses its private key to clean the noise and accurately recover the exact same symmetric key.

2.3.1.2 Practical Consequence for Software Design

Because of the transition from the Interactive Key Exchange to the one-way Key Encapsulation Mechanism (KEM), a significant architectural requirement is imposed on the development of hybrid protocols. As established in the previous section, ML-KEM operates under a different workflow compared to Diffie-Hellman, involving three distinct phases:

$$\text{KeyGen} \rightarrow \text{Encapsulate} \rightarrow \text{Decapsulate}.$$

To avoid architectural divergence within the hybrid protocol, an abstraction layer is implemented, which unifies the cryptographic paradigms. To achieve this in practice, a Wrapper pattern is applied, which unifies the disparate behaviours of classic and post-quantum algorithms into a single interface. Hence, Diffie-Hellman can be mathematically refactored to work as a KEM interface.

This abstraction ensures that the higher-level protocol logic remains algorithm-agnostic, so the handshake state machine does not need to distinguish between the commutative algebra of Elliptic Curves and the noisy geometry of Lattices. It is also very important to achieve the Defense-In-Depth

principle, since it allows the Key Combiner function to treat classical and post-quantum components as black boxes, merging the entropy from them by using the Hybrid Key Derivation Function (HKDF).

2.3.2 ML-KEM (CRYSTALS-Kyber, FIPS 203)

ML-KEM (originally known as CRYSTALS-KYBER) is the first Key Encapsulation Mechanism based on lattices which was standardized by NIST. The Module-LWE problem, previously explained, is the root of its security. Three core phases can be distinguished in its workflow:

1. Key Generation: A secret vector s and an error vector e (with small coefficients) are generated by the receptor. As mentioned before, the matrix A is the algebraic structure in which the security of the protocol relies on. However, generating and transmitting this matrix over the network would consume a very large number of kilobytes of bandwidth per handshake, hence the latency in the protocol would be unassumable. To solve this, an Extendable Output Function (SHAKE-128) is used in ML-KEM. Its workflow is as follows:
 - Seed generation: The server generates a random, cryptographically secure 32-byte seed denoted as ρ , instead of generating the entire matrix, as mentioned.
 - Matrix Expansion: The server takes the seed ρ and inputs it into the SHAKE-128 function, which outputs a continuous stream of bytes that form the set of polynomial coefficients of matrix A , once parsed.
 - Public Key Computation: Once A has been generated and stored locally in memory, the server computes the vector $t = A \cdot s + e$, which represents a point close to the lattice.
 - Key output: The public key $pk = (\rho, t)$ is transmitted over the network while the secret key (sk) only composed by the secret vector s is stored.
2. Encapsulation: The sender (client) receives the public key $pk = (\rho, t)$ and expands ρ with the hash function SHAKE-128 to reconstruct the same matrix $A = \text{SHAKE} - 128(\rho)$. Then, a new uniformly random 32-byte value is generated, denoted as m , where $m \leftarrow \{0,1\}^{256}$. This value is combined with a hash of the public key and processed through a hash function to derive both an intermediate message and the randomness used during encryption: $(m', r) = H(m || H(pk))$. The client can now compute the ciphertext c by combining the matrix A , the public vector t , the error vectors e_1, e_2 and the randomness r . The ciphertext is $c = (u, v)$, where:

$$u = A^T \cdot r + e_1 \text{ and } v = t^T \cdot r + e_2 + \text{Enc}(m')$$

Here, $\text{Enc}(m')$ denotes the encoding of the message m' into the polynomial ring, mapping the binary message into polynomial coefficients so that it can be embedded into the lattice structure and later recovered despite the noise. Finally, the shared key K is derived by hashing the processed message before together with the ciphertext c :

$$K = H(m' || H(c))$$

The ciphertext c is sent to the server.

3. Decapsulation: Upon receiving ciphertext $c = (u, v)$, the server utilizes its secret key s to decrypt it and recover the processed message. This involves removing the noise which was previously introduced during encapsulation through lattice-based operations, and it is done by computing:

$$v - s^T \cdot u = (t^T \cdot r + e_2 + \text{Enc}(m')) - s^T \cdot (A^T \cdot r + e_1) \approx \text{Enc}(m')$$

given that $t = A \cdot s + e$. An important point here is that the server ensures CCA security by re-encrypting the message:

$$(m'', r') = H(m' || H(pk))$$

and comparing it with the received ciphertext:

$$c' = \text{Enc}(pk, m''; r')$$

If this process is successful ($c' = c$), the server derives the shared secret K by applying the same hash function to the recovered message and the ciphertext:

$$K = H(m' || H(c))$$

To conclude this subsection, the overall goal of this ML-KEM aligns with classical key exchange mechanisms as Diffie-Hellman, despite in this case a different mathematical problem (lattices) is used. Both constructions allow two parties to establish a shared secret over an insecure channel.

The key difference between these two KEM's lies in the underlying hardness assumption: while Diffie-Hellman relies on the discrete logarithm problem, which is vulnerable to quantum attacks, ML-KEM is based on the Module-LWE problem, which is believed to be resistant to quantum adversaries. As a result, ML-KEM can be seen as a post-quantum analogue to Diffie-Hellman, preserving the same fundamental objective but also providing stronger security guarantees in the presence of quantum computing.

2.3.3 Security and Implementation Considerations

The theoretical robustness of lattice-based cryptography does not automatically guarantee security in a real-world software environment. Translating the mathematical operations of ML-KEM into executable code introduces critical vulnerabilities, primarily related to Side-Channel Attacks (SCA), which exploit physical information leaked computation instead of weakness in the mathematical problem.

To avoid these kinds of risks, it is crucial that the implementations always have the same execution time, ensuring that both the execution time and memory access do not depend on secret values. Additionally, Key Encapsulation Methods security relies on high-quality randomness for generating seeds, secret vectors and ephemeral keys. If these values have biased structures or are predictable, the security of the entire scheme is compromised.

2.4 Post-Quantum Digital Signatures

While key establishment mechanisms (KEMs) such as CRYSTALS-Kyber (ML-KEM) ensure confidentiality by establishing a shared secret over an insecure communication channel, they do not solve the challenge of identity authenticity. Without a cryptographic mechanism to verify the identity of the communicating parties, an unauthenticated KEM negotiation remains vulnerable to Man-in-the-Middle (MitM) attacks, where an adversary can intercept the public key exchange and establish separate encrypted sessions with both victims.

Utilizing a Hash-based Message Authentication Code (HMAC) is not an option to mitigate this vulnerability due to its reliance on symmetric cryptography, which has two main limitations during the initial handshake phase (Krawczyk & Eronen, 2010):

- An HMAC fundamentally requires a pre-shared key. At the start of the handshake phase, any secret must be negotiated between the client and the server; hence, it is not possible to execute the initial identity verification (EITCA Academy, 2025).
- HMAC schemes do not provide non-repudiation. As both parties share the same key, either party can generate a valid authentication tag (MAC). Consequently, it is not possible to prove to a third party who originated the message.

To address these limitations, secure protocols and Public Key Infrastructures rely on Digital Signatures (Bernstein et al., 2009). Unlike symmetric primitives, digital signatures apply a public-key architecture: the signer uses a private key to generate the signature, while the corresponding public key is distributed to the other party. This guarantees non-repudiation, ensuring that the signing entity cannot later deny an action performed by its private key.

Because the security of current standards such as RSA or ECDSA is based on mathematical problems vulnerable to Shor's Algorithm, new digital signature algorithms capable of resisting quantum computers are required for long-term security architectures (Shor, 1997; Mosca, 2018).

2.4.1 Role of Digital Signatures in Secure Protocols

Digital signatures are a fundamental component in secure communication protocols, providing three interrelated security properties essential for a secure protocol design (Hoffstein et al., 2008):

1. **Non-Repudiation:** As only one party possesses the private key, it cannot deny having signed a message. If both parties share the same key, non-repudiation would not be achieved.
2. **Integrity:** A modification to the signed message or any transmitted parameter leads to a failed verification process.
3. **Authentication:** It verifies that the message has been generated by an entity who possesses the secret key.

In the context of handshake protocols, digital signatures are employed to authenticate the handshake transcript—which includes the ephemeral key shares— (Stebila et al., 2020). This allows both parties to verify that the peer's public key originated from the corresponding identity. In the absence of digital signatures, impersonation attacks which compromise the entire key exchange mechanism could be performed arbitrarily (Bindel et al., 2019).

2.4.2 NIST-Selected Signature Schemes

In contrast to post quantum KEMs, where ML-KEM was formally established as a primary standard, in the field of digital signatures there are multiple standardized candidates. Among these candidates, NIST has selected multiple signature schemes; however, no single algorithm has been established as a universal replacement for classical digital signature algorithms due to distinct trade-offs in signature lengths, computational overhead and key sizes (NIST, 2024c; ETSI, 2017). In the following sections, two of these post-quantum signature frameworks are presented and analysed.

2.4.2.1 CRYSTALS-Dilithium

Similarly to Kyber, CRYSTALS-Dilithium (ML-DSA) is a lattice-based digital signature scheme designed to replace classical algorithms whose security is derived from the hardness of integer factorization or discrete logarithm problem. Dilithium has been selected within the NIST post-quantum cryptography standardization process and standardized as ML-DSA under the FIPS 204 publication (NIST, 2024b).

2.4.2.1.1 Mathematical Structure and the "Abort" Mechanism

The security of ML-DSA is based on the hardness of two geometric lattice-based problems: the Module Learning With Errors (M-LWE) problem, which ensures that recovering the private key from public data is computationally infeasible, and the Module Short Integer Solution (M-SIS) problem, which guarantees that generating a valid signature without knowledge of the secret key is also computationally unfeasible.

A foundational difference between classical public-key signature schemes and lattice-based post-quantum alternatives is the addition of random noise to mask the secret parameters. However, this requirement introduces a critical vulnerability: a possible statistical leakage of metadata regarding the secret key over a large transcript of signatures. To analyse this vulnerability, it is necessary to examine the lattice-based signature generation mechanics. Signing a message in CRYSTALS-Dilithium requires masking the secret vector s_1 by an ephemeral random noise vector y , producing a potential signature vector z :

1. An ephemeral public component w from the noise is computed via matrix multiplication:

$$w = A \cdot y$$

where $A \in R_q^{k \times l}$ is a public matrix formed by polynomials within the ring:

$$R_q = Z_q[X]/(X^n + 1)$$

and $y \in R_q^l$ is a vector composed of l ephemeral masking polynomials. As established in Section 2.2.3.3, the utilization of polynomials allows hundreds of coefficients to be compressed into a single algebraic element, which allows the use of a high security level while reducing both key and signature sizes.

2. The resulting vector $w \in R_q^k$ is decomposed into two distinct bit-planes: a high-bits vector (w_1) and a low-bits vector (w_0). This structural decomposition satisfies two structural constraints:
 - It increases the computational complexity for an attacker attempting to reconstruct the ephemeral mask y due to the missing low-bits vector.
 - It reduces the data size, thereby increasing the efficiency of the transmission process.
3. The high-bits vector is hashed together with the message m to compute the cryptographic challenge c :

$$c = H(m || w_1)$$

4. This challenge is used to calculate the potential signature vector z which can be formulated as:

$$z = y + c \cdot s_1$$

Because each signed message establishes a linear relation between the published signature and the secret key, a structural risk is introduced. An attacker can collect multiple signatures ($z_1, \dots, z_n$) generated by the same secret s_1 and perform statistical or lattice-based attacks over the entire set of signatures, since repeated signatures can introduce statistical dependencies, and the attacker can recover partial information about the secret key.

To prevent this secret key leakage, the Fiat-Shamir with Aborts framework is used. This mechanism relies on rejection sampling. Before publishing the potential signature z , this algorithm is used to evaluate its internal mathematical structure (the objective is to avoid that the potential signature z depends on the secret key s_1). In the Module-LWE problem, vectors like z, y and s_1 are composed of polynomials containing hundreds of discrete coefficients.

The algorithm performs a strict maximum-normalization test on these coefficients based on specific system parameters. The masking vector y is generated such that its coefficients do not exceed a maximum boundary defined by the system parameter γ_1 . Similarly, the influence of the secret term $c \cdot s_1$ is bounded by the system parameter β .

For the signature to be secure, every single coefficient in the resulting vector z must be strictly smaller than the threshold $\gamma_1 - \beta$. By using this bound it is ensured that the random noise y dominates the distribution of the potential signature z , limiting the contribution of the secret term $c \cdot s_1$. If even one coefficient equals or exceeds this limit, it means the random noise y could not securely mask the secret s_1 . Additionally, the low-bits component w_0 is bounded by γ_2 . If any of these bounds are crossed, the algorithm immediately discards the potential signature, deletes the used mask, generates a completely new random noise vector and restarts the calculation. This loop repeats until a safe signature is produced, ensuring that every published signature resists the statistical analysis threat.

2.4.2.1.2 Algorithmic Workflow

Once established the mathematical foundations required to understand this scheme, the algorithm's design can be analysed. Its workflow is decomposed in three different algorithms:

1. Key Generation: It is very similar to the ML-KEM. To avoid sending a large matrix A , the signer generates a 32-byte seed ρ and extending it with the SHAKE-128 function, where each entry of the matrix A is a polynomial in the ring $R_q = Z_q[X]/(X^n + 1)$ with $n = 256$. The public vector t which is one of the components of the public key pair is computed as:

$$t = A \cdot s_1 + s_2$$

where s_1 and s_2 are the secret vectors. The public key pair (ρ, t) is published while the secret key (sk) is hidden.

2. Signature Generation: The random noise vector y of polynomials is generated, with its coefficients bounded by γ_1 . The public component w is computed as $w = A * y$ and then each component of it is decomposed on high bits (w_1) and low bits (w_0) such that:

$$w = w_1 \cdot 2\gamma_2 + w_0 \text{ and } |w_0| \leq \gamma_2$$

A cryptographic challenge c is computed by hashing the high bits w_1 and the message m :

$$c = H(m, w_1)$$

Its utility is to bind the signature to the message and the random noise vector y , ensuring that each signature is unique and it cannot be reused for signing a different message. Then the potential signature vector is calculated:

$$z = y + c \cdot s_1$$

Now the abort mechanism is applied as explained before. Each coefficient of the vector z must satisfy that

$$|z_i| < \gamma_1 - \beta$$

If this threshold is violated, the whole process is restarted with a new y vector. Finally, the pair (z, c) and the message m are transmitted to the receiver. The public key (ρ, t) is also available to the verifier, it can be sent together with the message and the signature or obtained before this process.

3. Verification: The verifier receives the message m , the pair (z, c) and the public key (ρ, t) . As the seed ρ is public, the same matrix A can be computed by using the XOF SHAKE-128. Then the verifier computes:

$$u = A \cdot z - c \cdot t$$

and by using that:

$$z = y + c \cdot s_1 \text{ and } t = A \cdot s_1 + s_2$$

$$u = A \cdot y - c \cdot s_2$$

which is very similar to $w = A * y$. The perturbation introduced by the term $-c * s_2$ only affects the low-bits of the coefficients of u ; therefore, the high-bits vector (denoted as w'_1) remain unchanged. These high bits are extracted from u , since each component of u can be decomposed as:

$$u_i = w'_{1,i} \cdot 2\gamma_2 + u_{0,i}$$

where $w'_{1,i}$ represents the high bits of u . Finally, w'_1 is used to recompute the cryptographic challenge $c' = H(m, w'_1)$, which is accepted only if $c' = c$ and the boundary conditions explained before are satisfied. The detailed algebraic proof of correctness and polynomial substitution showing how the terms cancel out during verification is provided in Appendix B.1

2.4.2.2 Falcon

Falcon (Fast Fourier Lattice-based Compact Signatures) is an alternative lattice-based digital signature scheme, selected in the NIST post-quantum cryptography standardization process. It is underpinned by Gaussian sampling over structured lattices and its security is derived from the computational hardness of solving the Short Integer Solution (SIS) and Nth-degree Truncated Polynomial Ring Units (NTRU) problems, while its efficiency comes from the utilization of Fast Fourier Transform techniques in order to accelerate polynomial operations (Fouque et al., 2018).

2.4.2.2.1 NTRU Lattices Structure

Falcon operates over NTRU lattices, which are special mathematical structures that enable signatures generation securely and efficiently. The main idea is that while within this geometric structure, an infinite grid of vectors exists, only a restricted subset of short vectors with small coefficients are valid for producing a signature.

The architectural paradigm rests on the private key acting as a cryptographic trapdoor. This private key allows the signer to isolate short vectors by providing specific geometric basis information, and at the

same time, finding such short vectors is computationally infeasible without possessing the secret key. Recovering these short vectors is what enables secure signature generation and verification.

2.4.2.2.2 Gaussian Sampling in Falcon

The main difference between CRYSTALS-Dilithium and Falcon lies in the coefficient distribution during signature generation. While Dilithium applies rejection sampling and samples coefficients uniformly, Falcon uses a Gaussian distribution to sample vector components, to avoid any statistical or transcript-based leakage of metadata regarding the private key (Lyubashevsky, 2012).

In practice, this signing process requires generating short vectors according to a Gaussian distribution centered around a target derived from the hashed message. However, performing this sample directly in the lattice domain is computationally expensive and inefficient due to the existent dependencies between the coefficients of the polynomials,

To address this limitation, the algorithm implements Fast Fourier Transform (FFT), since it works on the frequency domain of the polynomials instead of the coefficient domain. By doing this the computational complexity is reduced because convolutions are replaced by point-wise multiplications. Also, FFT enables a representation of the lattice in which its components can be handled almost independently, making it possible to efficiently apply techniques such as Gaussian sampling (Ducas & Prest, 2013).

2.4.2.2.3 Algorithmic Workflow

The primary objective of the Falcon signing infrastructure is to compute a short lattice vector that approximates the target vector while ensuring that no information about the secret key is leaked. As in Dilithium, the operational workflow is organized in three distinct phases:

1. Key Generation: Two small polynomials f and g are generated, forming a short basis of the lattice. Together they form the secret key. From these parameters, the public key h is computed as:

$$h = g \cdot f^{-1} \bmod q$$

needed for verification. The public key h is distributed to allow anyone to verify a signature, keeping secret the trapdoor.

2. Signature generation: This process is divided into several steps:
 - Hashing the message: Given a message m , a hash function is applied to produce a digest $c = H(m)$. It is important not to confuse this parameter with the challenge c that is computed in Dilithium. This digest ensures that the signature is bound to the message. It is mapped into the polynomial ring $R = \mathbb{Z}[X]/(x^n + 1)$, resulting into the target vector t .
 - Frequency domain transformation: As noted previously, the Fast Fourier Transform is first applied on the secret lattice basis to work over the frequency domain, enabling efficient Gaussian sampling.
 - Secret basis decomposition: Once the secret basis has been transformed on the frequency domain, they are decomposed to apply recursive sampling of each coordinate independently. This is done by using LDL decomposition.
 - Gaussian sampling: In each coordinate it is sampled a value from a discrete Gaussian distribution centered on the mean derived from the target vector t and the trapdoor. The resultant vector avoids any leakage of the secret key since its distribution ensures statistical indistinguishability.
 - Inverse FFT: The sampled vector is transformed back into the coefficient domain using the inverse FFT. Also, rejection sampling is applied (each coefficient is verified against the norm bound).
 - Output: Once a valid vector z is obtained, it constitutes the Falcon signature for the message. The public key h is available to the verifier for validation.

3. Verification: The verifier receives the public key h , the signature z and the message m . The message is hashed obtaining the target value $c = H(m)$, which is then mapped into the ring. Using the public key h the verifier checks that:

$$s_1 + s_2 \cdot h \equiv c \text{ mod } q$$

and that z is close enough to the target t . Finally, the signature is accepted if the distribution is consistent with the Gaussian sampling procedure and all the coefficients satisfy the norm bounds.

2.4.3 Comparative Analysis of PQ Signature Schemes

These digital signature schemes offer alternative mechanisms to classical schemes like RSA or Elliptic Curve Digital Signature Algorithm (ECDSA) while providing security against quantum computing. To evaluate both systems and decide which one suits better for this practical implementation the following table compares CRYSTALS-Dilithium and Falcon across key factors relevant for cryptographic security and efficiency:

Table 1: Architectural and Structural Comparison of Post-Quantum Signature Primitives.

Feature Criterion /	CRYSTALS-Dilithium (ML-DSA-2)	Falcon (Falcon-512)	Primary Source
Underlying lattice	Module-LWE / Module-SIS	NTRU lattices / SIS	NIST FIPS 204; Fouque et al. (2018)
Sampling method	Uniform + rejection sampling	Discrete Gaussian + rejection sampling	NIST FIPS 204; Falcon Spec
Signature size	2420 bytes	666 bytes	NIST FIPS 204; Falcon Spec
Public key size	1312 bytes	897 bytes	NIST FIPS 204; Falcon Spec
Algorithmic & Implementation complexity	Low/Moderate: Relies on standard polynomial arithmetic; avoids floating-point operations and FFT.	High: Requires FFT and high-precision floating-point arithmetic.	Ducas & Prest (2013); NIST PQC Reports
Standardization status	Selected and standardized by NIST, FIPS 204	Selected by NIST, widely studied but pending FIPS finalization	NIST (2023–2024)

To conclude, while both signature schemes provide strong post-quantum security based on lattice-based schemes, they target fundamentally distinct optimization criteria. CRYSTALS-Dilithium offers a balanced trade-off between security and implementation simplicity, relying on uniform sampling combined with rejection sampling. No Fast Fourier Transform (FFT) is applied, as standard polynomial arithmetic suffices for both key generation and signing. Conversely, Falcon provides strong protection against statistical leakage due to the use of Gaussian sampling but increases the implementation complexity since FFT is required to produce signatures securely, as well as recursive decomposition.

Considering these structural trade-offs and the objectives of this research, CRYSTALS-Dilithium is the more suitable scheme for the practical part of this work due to its ease of implementation, lower risk of errors and its sufficient security.

2.5 The Mitigation Strategy: Hybrid Cryptography

Although post-quantum cryptographic algorithms are designed to resist quantum attacks, they have limited deployment experience compared to well-established classical algorithms. For that reason, exclusive reliance on them presents potential risks related to implementation vulnerabilities or future advances in mathematical theory. To mitigate this transitional risk, the current solution is the adoption of the hybrid cryptography model, which combines both classical and post-quantum primitives within a single protocol session (Stebila et al., 2020).

2.5.1 The "Harvest Now, Decrypt Later" Threat:

Even though quantum computers are expected to be available in the next decade, the urgency of implementing post-quantum algorithms in current systems comes to achieve long-term data confidentiality. The "Harvest Now, Decrypt Later" (HNDL) threat relies on adversaries intercepting and storing network traffic transmitted today, with the purpose of decrypting it once quantum computers can break classical primitives. This explains the urgency of a fast transition to post-quantum cryptography.

2.5.2 The Hybrid Paradigm:

The architecture of the hybrid model relies on the parallel execution of both a classical key exchange with a post-quantum scheme within the same key establishment phase. In this mode, each scheme produces its own shared secret.

During a handshake session, each primitive independently computes its respective shared secret. These secrets are concatenated and processed through a key derivation function (KDF) to derive the final session key. With this approach, the resulting derived key remains secure as long as one of the underlying schemes is not compromised.

2.5.3 Cryptographic Combiners and Deployment Asymmetry

This principle is similarly applicable to both authentication and digital signatures, which must be considered in secure protocols since they provide integrity, authenticity and non-repudiation. As highlighted earlier, the idea of hybrid schemes is to avoid that the overall security of the whole protocol depends on a single assumption, establishing cryptographic combiners at two different levels:

1. The Key Establishment Phase: Classical and post-quantum mechanisms are combined to derive a shared secret.
2. The Authentication Layer: Where a classical digital signature scheme (ECDSA) is combined with a post-quantum scheme (CRYSTALS-Dilithium/ML-DSA), generating a signature pair. Verification requires both signatures to be valid since a message is signed using both signature schemes. Hence, even if the classical scheme becomes vulnerable by quantum computation in the future, security is still guaranteed by the post-quantum component.

However, it is critical to note that production frameworks and commercial standards such as TLS 1.3 have prioritized the immediate integration of hybrid KEMs over hybrid digital signatures schemes. This is driven by the fact that quantum authentication attacks can only be executed in real-time once a physical computer is fully operational, whereas the HNDL threat targets data confidentiality retroactively. Consequently, updating the KEMs protects historical data from future decryption, whereas updating the signature layer can be deferred until relevant quantum computers are physically deployed. This operational distinction serves as the theoretical foundation for the empirical evaluations and structural bottlenecks analysed in the practical phase of this work.

Chapter 3

Practical Part:

Leaving aside the theoretical part, this chapter presents the practical implementation of a hybrid cryptographic system designed to mitigate the “Harvest Now, Decrypt Later” (HNDL) threat model, previously explained in Chapter 2. The principal motivation is to link the previously explained foundations with their application in a real, functional environment. By doing so, the main goal of this implementation is to demonstrate how classical compatibility and post-quantum security can be achieved at the same time through a hybrid design. Consequently, a hybrid design is detailed in the following sections, focusing on the immediate protection of confidentiality and authentication.

3.1 Proposed Architecture: The Hybrid Paradigm

The protocol implements a layered, block-oriented architecture divided into a primitive’s layer (cryptographic building blocks) and a protocol layer (handshake composition). This design separates session logic from the actual algorithm implementations. Defense-in-Depth (Barker & Kelsey, 2023) and mitigation of “Harvest Now, Decrypt Later” threats require that the architecture execute classical and post-quantum algorithms in parallel utilizing two independent mathematical assumptions:

- Hybrid Key Exchange: Elliptic Curve Diffie-Hellman (ECDH) and Module-Lattice-based Key Encapsulation Mechanism (ML-KEM/Kyber) operate independently producing two shared secrets. These secrets are then concatenated and processed via an HMAC-based Key Derivation Function (HKDF) to derive the final session keys (Krawczyk & Eronen, 2010).
- Hybrid Authentication: To establish Integrity and Authenticity, a dual-signature method is utilized in this architecture. That is accomplished once again through the combination of a Classical (ECDSA) and a Post-Quantum Algorithm (ML-DSA/Dilithium) to generate a Composite Signature Package (Bindel et al., 2019). The Verification Process is fulfilled only if both Signatures have been validated; therefore, an Adversary would have to break both Schemes to Compromise the Security of the Protocol.

3.1.1 Defense-In-Depth Principle

The core of this project is the Defense-in-Depth principle. This principle stipulates that the security of a communication channel cannot depend on a single point of failure (Stebila et al., 2020). Thus, the protocol security is derived from two structurally different and independent mathematical assumptions:

- Classical Layer: Based on the computational infeasibility of the Discrete Logarithm Problem (DLP), upon which Elliptic Curve Diffie-Hellman bases its security (Paar & Pelzl, 2010).
- Post-Quantum Layer: Based on the hardness of lattice-based problems (Module-LWE concretely), which underpins ML-KEM/Kyber and ML-DSA/Dilithium security (NIST, 2024).

These layers are implemented in both key exchange and authentication mechanisms, minimizing the risks related to the implementation of recently standardized post-quantum primitives while ensuring long-term confidentiality against the HNDL threat.

3.2 Primitives (Building Blocks)

The practical implementation of the hybrid handshake protocol is structured in incremental steps, focusing on developing a solid theoretical base for each primitive and building blocks prior to addressing high-level performance. Therefore, to comprehend the combination of classical and post-quantum mechanisms, reliance on external libraries is minimized as much as possible, except for post-quantum modules. This approach provides an analysis of how primitives operate and interact before integrating it into the complete protocol.

3.2.1 HMAC & SHA-256

This section details the Hash-based Message Authentication (HMAC) implemented over the SHA-256 hash algorithm, utilized in this hybrid protocol to ensure both the integrity and authenticity of the exchanged data.

3.2.1.1 Cryptographic Role and Vulnerability Mitigation

HMAC has a dual purpose within the proposed architecture: it serves as a pseudorandom function within the Key Derivation Function (HKDF) and provides both message integrity and source authentication once the session key is established (Krawczyk et al., 1997). While both message authentication codes (MACs) and digital signatures ensure that data has not been altered in transit, MACs use a shared symmetric key k to generate and verify an authentication tag. Its reliance on symmetric hash operations makes MACs significantly faster than any asymmetric alternative, minimizing overhead (Paar & Pelzl, 2010).

The security of this primitive is based on SHA-256, a cryptographic hash function that maps an input of arbitrary length to a fixed 256-bit (32-byte) digest resistant to pre-image, second pre-image and collisions attacks (NIST, 2015). These properties make it suitable for use in HMAC and HKDF, where a reliable and unpredictable hash function is needed. However, it is important to note that in standard hash constructions such as secret prefix MACs $m = H(k||x)$ Merkle–Damgård based functions are vulnerable to length extension attacks (Katz & Lindell, 2020), since these functions process the input message iteratively, block by block (all blocks of the same size), the final hash output corresponds exactly to the internal state of the hash function after processing the entire message. This design choice enables an attacker to treat a valid hash output as a valid internal state and continue hashing additional data without knowing the original input.

Let a sender compute a secret-prefix MAC for a message $x = (x_1, x_2, \dots, x_n)$ as:

$$m = H(k||x_1, x_2, \dots, x_n)$$

An adversary who intercepts both the message x and its corresponding MAC m can exploit the Merkle–Damgård structure by appending an additional block x_{n+1} to the message. Without knowledge of the secret key k , the attacker can compute:

$$m' = H(m||x_{n+1})$$

This newly computed value m' is interpreted by the receiver as $H(k||x_1, x_2, \dots, x_n, x_{n+1})$, which represents the valid MAC of the extended message. Consequently, the receiver will accept the modified message as authentic, even though the sender never authorized the appended data. Unlike common hashing, HMAC solves these vulnerabilities thanks to its nested structure based on double hashing (Krawczyk et al., 1997), guaranteeing that a valid internal state can be only computed if the secret key is known.

3.2.1.2 Algorithmic Structure and Workflow

The HMAC algorithm combines a cryptographic hash function (SHA-256) with a secret key to ensure a compatible length-fixed key which guarantees integrity and authenticity through the following workflow:

1. **Key Normalization:** The key is normalized to the block size by zero-padding if it is shorter than the block size or hashing it down if it is longer. This is required since the key must be exactly as long as the internal block of the hash to ensure consistency.
2. **Mask Generation:** Two masks (i_{pad} and o_{pad}) are created and XORed with the key to compute two different versions of the original secret key, which allows the algorithm to perform two hashes (inner and outer) using different keys. In SHA-256, i_{pad} consists of the byte 0x36 repeated exactly 64 times, while o_{pad} consists of the byte 0x5c repeated 64 times.
3. **Nested Evaluation:** An inner hash is computed by hashing the concatenation of the key XORed with i_{pad} and the message:

$$H_{inner} = H((K \oplus i_{pad})||M)$$

The outer hash (which is the same as the final HMAC value) is computed by hashing the concatenation of the key XORed with o_{pad} and the inner hash:

$$HMAC(K, M) = H_{outer}(K, M) = H((K \oplus o_{pad}) || H_{inner})$$

Upon receipt, the receiver recomputes the HMAC by using the secret key to verify that it matches the received value.

3.2.1.3 Practical Implementation Analysis

In this practical core, the Python **hashlib** library is used to compute the SHA-256 primitives rather than a custom-built implementation. Implementing the 64 rounds of compression increases code complexity, without providing a significant structural value toward understanding how a handshake protocol works, which is the main goal of this project. Since SHA-256 is not the focus of this project, it is better to use a python library. However, it is important to understand how SHA-256 works internally, as it is the core of both HMAC and HKDF functions, understanding its internal workflow remains necessary to prevent remote leakage and analyse protocol vulnerabilities. The technical steps regarding the workflow of SHA-256 are detailed in Appendix C.1.

3.2.2 HMAC-based Key Derivation Function (HKDF)

A Key Derivation function (KDF) is required to transform the shared secret obtained from asymmetric key exchanges into cryptographically strong key material. The raw outputs generated from these schemes are unsuitable as symmetric keys since they do not guarantee uniform randomness; often exhibiting algebraic structures, mathematical bias or non-standard lengths which can be exploited by an attacker (Krawczyk, 2010).

3.2.2.1 Design

To mitigate the structural vulnerabilities mentioned before, the protocol implements HKDF (HMAC-based Key Derivation Function), which extracts the entropy of the initial key material and returns one or more keys (Krawczyk & Eronen, 2010). Its operational design is divided into two phases:

- Extract phase (**hkdf_extract**): The algorithm takes the Input Key Material (*IKM*) representing the raw shared secret and applies HMAC operations using a salt to remove possible algebraic structure or bias, producing a fixed-length Pseudorandom Key (*PRK*):

$$PRK = HMAC.Hash(Salt, IKM)$$

- Expand phase (**hkdf_expand**): It derives the final output keying material from the pseudorandom key obtained during the extract phase.

3.2.2.2 Implementation & Analysis

The security of the keys derived through HKDF is based on the implementation of both extract and expand phases. Three important parts of the code are crucial to the security of the keys derived from the previous secret:

1. Salt Handling: The **hkdf_extract** function handles cases where the salt is not provided. If the salt parameter results in None, it defaults to a string of null bytes with the length of the hash output (32 bytes) to ensure that the underlying HMAC function always operates with a valid key structure, even when no external entropy is available. The salt allows to transform the input key material into a pseudorandom key, removing all possible algebraic structure or bias contained in the previously computed secret.
2. Iterative Expansion: The **hkdf_expand** function implements a cryptographic feedback loop. Inside the generation loop, the previous block is concatenated into the input of the next HMAC calculation. This ensures that every block of the output key material is mathematically dependent on the previous ones, maintaining high entropy across the entire length of the generated key.
3. Context Binding: The info parameter allows for domain separation. By including this specific context string in the hash input, the derived key is bound to a specific protocol phase. This

ensures that even if the same initial secret is reused, different info values will result in completely different session keys, preventing cross-context attacks.

3.2.3 X25519 (Classical Key Exchange)

This section focuses on the integration of the X25519 scalar multiplication into an ephemeral Key Encapsulation Mechanism (KEM) framework by processing the resulting raw shared secret through a Key Derivation Function (HKDF).

3.2.3.1 KEM Design

To homogenize software interfaces with post-quantum primitives, X25519 is structured as an ephemeral KEM. Both client and server generate ephemeral key pairs for every session, rejecting any use of static or long-term exchange keys on the server side, to achieve forward secrecy.

The pipeline runs as follows:

1. Key Generation (**kem_keygen**): Executed dynamically by the server when receiving a handshake request to generate an ephemeral public/private key pair for the current session.
2. Encapsulation (**kem_encapsulate**): The client generates an ephemeral key pair to compute a raw shared secret using the server's ephemeral public key, then outputs its public key, which is derived into the session key component via HKDF.
3. Decapsulation (**kem_decapsulate**): The server executes reciprocal scalar multiplication using its ephemeral private key and the client's public key, re-running the HKDF to derive the identical session key.

By using this architecture, the authentication part is completely isolated from the KEM, since the server establishes identity exclusively by signing the session transcript using its static digital signatures.

3.2.3.2 Implementation & Analysis

While section 1.2.3 introduced Elliptic Curve Cryptography (ECC) using full-point arithmetic and general Weierstrass equations, this implementation uses Curve25519 together with the X25519 Diffie-Hellman function (Paar & Pelzl, 2010).

Curve25519 is a Montgomery curve defined over the prime field F_p with $p = 2^{255} - 19$, expressed as: $y^2 = x^3 + Ax^2 + x \bmod p$ where $A = 486662$. Unlike the geometric rules explained in Section 1.2.3.1, X25519 performs scalar multiplications using only the x-coordinate, simplifying finite-field arithmetic and reducing bandwidth.

The software module operates without relying on external cryptographic libraries, but rather on the functions **dh_generate_private_key** and **dh_generate_public_key**. To ensure unpredictable outputs, a cryptographically secure pseudo-random number generator outputs 32 random bytes and is used together with X25519 clamping to enforce curve-specific constraints (detailed in Appendix A.4). Public parameters are computed via **x25519_scalar_mult** function, which restricts all modular calculations within the field defined by $p = 2^{255} - 19$.

3.2.4 Kyber/ML-KEM (Post-Quantum Key Exchange)

A post-quantum Key Encapsulation Mechanism (KEM) runs in parallel to provide security against both classical and quantum adversaries capable of running Shor's algorithm.

In total alignment with the classical structure explained above, this post-quantum layer rejects the use of static parameters for the KEM. Both modules operate as ephemeral primitives, i.e. all public keys are discarded once the session key is derived. The subsequent HKDF key derivation and transcript authentication stages remain unchanged, as both process the secrets independently of how that symmetric material is established.

3.2.4.1 Design

Exactly as the classical building block established in the previous section (X25519 as a KEM), the ML-KEM-512 variant is structured around three logical phases (already explained with more detail in Section 2.3.2):

1. Ephemeral Key Generation (**kyber_keygen**): As in the classical KEM, it is executed by the server upon receiving a handshake request to generate a short-term key pair. This practical stage computes the matrix expansion seed ρ together with the resulting ephemeral public vector $t = A \cdot s + e$, which composes the single-use public key for the current session.
2. Ephemeral Encapsulation (**kyber_encapsulate**): The client performs a probabilistic encapsulation using the server's ephemeral public parameters. A 32-byte seed m is generated and encrypted into a ciphertext $c = (u, v)$ using the expanded matrix A and the public vector t . Each encapsulation generates distinct randomness, producing a unique ciphertext for every session.
3. Decapsulation (**kyber_decapsulate**): The server receives and processes the client's ciphertext $c = (u, v)$ using its private vector s to compute $v - s \cdot u$, cancelling out the lattice noise to recover the identical seed m .

3.2.4.2 Implementation & Analysis

This practical implementation integrates the FIPS 203 compliant module **kyber_py**, whose implementation can be summarized in three concrete functions: **kyber_keygen**, **kyber_encapsulate** and **kyber_decapsulate**.

By forcing both **kyber_keygen** and **kyber_encapsulate** functions to be executed every handshake, the protocol ensures that single-use parameters are generated by both the server and client. This symmetric distribution of parameters avoids key reuse vulnerabilities, successfully achieving Perfect Forward Secrecy (PFS) across both cryptographic layers.

3.2.5 ECDSA/secp256r1 (Classical Digital Signature)

While the key exchange mechanisms (X25519 and ML-KEM) operate raw and ephemerally to ensure forward secrecy, they do not verify identity. ECDSA is the classical component of the authentication layer, and it uses secp256r1 curve to avoid man-in-the-middle attacks. Its 64-byte size signature reduces network overhead while maintaining an optimal performance baseline.

3.2.5.1 Design

The authentication framework follows a similar structure to the previous blocks, with the only difference being that it transitions from an ephemeral key agreement scheme to a static authentication mechanism.

- Static Key Generation: Over a curve group defined by the modulus p , coefficients a, b and a base point G of prime order q , the server generates the private key d as a random integer $0 < d < q$ and derives its static public key B through scalar multiplication: $B = d \cdot G$.
- Transcript Signing: The server collects all ephemeral public keys and nonces exchanged until that point into a transcript, computing the hash digest $e = \text{SHA256}(m)$. By using an ephemeral nonce $k \in (0, q)$, it generates the signature pair (r, s) by computing $r = x_R \bmod q$, where $R = k \cdot G$ and $s = k^{-1}(e + d \cdot r) \bmod q$.
- Signature Verification: The client recomputes the transcript digest $e = \text{SHA256}(m)$ and computes the intermediate value $w = s^{-1} \bmod q$. The point r is reconstructed as $R' = u_1 \cdot G + u_2 \cdot B$, where the auxiliary values u_1, u_2 are calculated as $u_1 = e \cdot w \bmod q$ and $u_2 = r \cdot w \bmod q$. The signature is only valid if $x_{R'} \equiv r \bmod q$.

3.2.5.2 Implementation & Analysis

While a custom implementation was relatively feasible for X25519, digital signatures are highly sensitive to side-channel attacks and nonce entropy. Consequently, the cryptography library is used to execute secp256r1 primitives, avoiding remote leakage.

The **generate_keypair** function establishes the server's identity for classical authentication using ECDSA with the NIST P-256 (secp256r1) curve, providing approximately 128 bits of classical security. The **sign** and **verify** functions translate this design into a session-bound mechanism where ECDSA signatures authenticate ephemeral public keys exchanged during the handshake. To prevent private key leakage, SHA-256 is used as the hash algorithm, and deterministic ECDSA (RFC 6979) is used to prevent signature randomness attacks, generating the resulting signature in DER format for standardization.

3.2.6 Dilithium/ML-DSA (Post-Quantum Digital Signature)

To protect the handshake against quantum authentication vulnerabilities, ML-DSA is integrated into the protocol as the post-quantum component of the authentication layer, operating in parallel with ECDSA. While its security is based on lattice-based problems, specifically the Module Learning With Errors (MLWE) and Module Short Integer Solution (M-SIS) problems explained in Section 2.4.2, its practical implementation is characterized by the use of rejection sampling, which avoids statistical attacks to infer information about the signer's secret key. This is done by an internal iterative loop, which accepts or discards the parameters generated depending on boundary conditions.

3.2.6.1 Design

The authentication framework follows a similar structure to the previous blocks, with the only difference being that it transitions from an ephemeral key agreement scheme to a static authentication mechanism.

- **Static Key Generation:** The server generates a long-term, post-quantum key pair. The static public key is composed of the seed ρ and the polynomial vector:

$$t = A \cdot s_1 + s_2$$

where A is the matrix expanded from the seed ρ and s_1 and s_2 the secret vectors.

- **Transcript Signing and Rejection Sampling:** The server uses its private key to sign the accumulated session transcript m , computing the cryptographic challenge

$$c = H(m, w_1)$$

and it samples a masking vector y to compute the potential signature

$$z = y + c \cdot s_1$$

Before signature transmission, it is checked that the boundary conditions are satisfied by executing the rejection sampling loop until these conditions are satisfied. The published signature is the pair (z, c) , which is independent from the secret key s_1 .

- **Signature Verification:** The client receives the signature (z, c) and uses the server's public key to reconstruct the high-bits vector w_1' by computing

$$u = A \cdot z - c \cdot t$$

Following the workflow explained in Section 2.4.2.1.2 the client recomputes the challenge:

$$c' = H(m, w_1')$$

If c' matches the received challenge c and all boundary conditions are satisfied, the signature is accepted. This confirms that the identity of the server is legitimate.

3.2.6.2 Implementation & Analysis

In this practical implementation it is used the **dilithium_py.ml_dsa** library, which is divided into three functions: **generate_keypair**, **sign**, and **verify**, keeping architectural symmetry with the classical ECDSA layer.

The choice of ML-DSA-44 ensures a 128-bit quantum security level comes to ensure the same strength of Kyber-512 and X25519. Although the signature size (~2.4 KB) is much larger than in classical schemes, the computational overhead is minimal due to efficient lattice arithmetic. A key security feature is the deterministic signing process, which protects the protocol against vulnerabilities in random number generators. By signing the full handshake transcript, the implementation achieves Identity

Binding, ensuring the server's long-term identity is cryptographically tied to the ephemeral session parameters.

3.3 Protocol Construction

Once all primitives have been defined and explained in depth, the final step of this project is to integrate all of them to construct the handshake protocol. As discussed earlier, the main goal of this protocol is to establish a secure communication channel between the server and client that guarantees both confidentiality and authenticity security, even against quantum computation.

This section bridges all the theoretical foundations explained in Chapter 1&2 (classical Diffie-Hellman, elliptic curves, post-quantum KEMs as Kyber, signature schemes as Dilithium) with their practical implementations detailed in Section 3.2 (X25519, ML-KEM, ML-DSA), which results in a functional handshake flow. This protocol construction is divided into three subsections:

1. Section 3.3.1 (Hybrid Key Exchange Construction): Focuses on how classical and post-quantum secrets are generated and combined to satisfy the Defense-in-Depth principle.
2. Section 3.3.2 (Hybrid Authentication Construction): Details how the authentication is achieved by again using a classical and a post-quantum signing algorithm.
3. Section 3.3.3 (Authenticated Hybrid Handshake Protocol): Integrates these components all together to form the complete Authenticated Hybrid Handshake and analyses its security properties.

3.3.1 Hybrid Key Exchange Construction

The hybrid key exchange represents the core contribution of this practical work. The framework derives its final session key from two independent shared secrets: one produced by a classical KEM (X25519) and one produced by the post-quantum KEM (ML-KEM-512). In alignment with the Defense-in-Depth principle, the underlying system security remains guaranteed as long as at least one of the two underlying assumptions holds (Stebila et al., 2020).

3.3.1.1 Design

The design of the hybrid key exchange follows a parallel composition model, as stated before. Both the client and the server initiate and execute a X25519 and a ML-KEM-512 flow simultaneously, where each output is completely isolated from the other.

By keeping both mechanisms separate until the final derivation step, several key operational advantages are achieved:

- Isolation of Security Reasoning: A structural failure of one primitive does not compromise the security of the other.
- Easier Architectural Maintenance: Any modifications to X25519 module do not affect ML-KEM implementation and vice versa, facilitating easier auditing.
- Future Scalability: Protocol upgrades are easier to implement in this model, since a component can be replaced without redesigning the entire flow.

3.3.1.2 Implementation & Analysis

Once both asymmetric channels have executed their respective scalar multiplication and decapsulation operations, the client and server isolate two distinct shared secrets: a 32-byte classical elliptic curve secret, denoted as ss_{X25519} and a 32-byte post-quantum kyber secret, denoted as ss_{ML-KEM} . These parameters are combined at the symmetric layer by using the HKDF engine (Krawczyk & Eronen, 2010), to bind them into a unified cryptographic state. The input to the key derivation function can be expressed as:

$$IKM = ss_{X25519} || ss_{ML-KEM}$$

This combined input is then processed using the standard HKDF extract-and-expand construction to derive a fixed-length Pseudorandom Key (PRK):

$$PRK = HKDF.Extract(Salt, IKM)$$

Although ML-KEM introduces a fundamentally different mathematical foundation compared to classical X25519, its decapsulated output fits naturally into the existing symmetric cryptographic pipeline. Because the post-quantum Kyber secret is a uniform byte array with sufficient entropy, it becomes suitable as input key material for standard key derivation functions.

This design choice highlights an important practical insight regarding post-quantum cryptography transition: the integration of lattice-based schemes primarily affects the asymmetric layer, while the symmetric layer remains structurally unchanged. Both HKDF and HMAC modules operate on raw key material without making assumptions about how that material was generated, if it is secret and unpredictable. The HKDF removes any statistical bias, or mathematical properties present in the independent shared secrets.

From a security standpoint, this parallel construction provides robustness: if either ss_{X25519} or ss_{ML-KEM} is indistinguishable from random, the resulting session key will maintain its cryptographic integrity while preventing data compromise under the HNDL threat. This property is central to the appeal of hybrid key exchange mechanisms. Finally, a KEM validation script was developed to ensure that this parallel setup works as designed and both nodes derive identical hybrid session keys. This verification test is detailed in Appendix C.4.

3.3.2 Hybrid Authentication Construction

This section details the implementation of the hybrid authentication layer, which guarantees the integrity of the server and, by consequence, of the handshake. This is done by implementing a dual-signature framework that concurrently executes a classical ECDSA instance and a ML-DSA instance, both explained in Section 3.2 (Bindel et al., 2019).

3.3.2.1 Design

The hybrid authentication, as well as the hybrid key exchange mechanism, follows a parallel composition model. The most important part of the authentication with digital signatures is the Dual Signature verification or the Composite Signature construction. When the server needs to authenticate the handshake, it executes both signing processes over the same data block simultaneously. Instead of signing a static message, a dual signature is generated by the server over a Session Transcript. This transcript is a hash of all ephemeral or public keys, nonces and known parameters exchanged during the handshake until that point:

$$T = SHA - 256(pk_{X25519_C} || pk_{KYBER_C} || N_C || pk_{X25519_S} || pk_{KYBER_S} || N_S)$$

where N_C and N_S are the Nonces generated by the server and the client.

By including this component in the protocol, it is ensured that an attacker cannot perform an attack by intercepting the message and modifying any parameter, since it would lead into a change in the hash and hence, a failure in the signature verification process (Bhargavan et al., 2016).

To satisfy the authentication process (at the client side), both classical and post-quantum signatures must be valid. The Hybrid validation formula is composed by the conjunction of both verification operations:

$$V_{hybrid} = V_{ECDSA}(pk_{ECDSA}, T, \sigma_{ECDSA}) \wedge V_{ML-DSA}(pk_{ML-DSA}, T, \sigma_{ML-DSA})$$

This mechanism achieves Identity Binding by cryptographically linking the server's long-term identity to the specific ephemeral parameters of the session, effectively neutralizing Man-in-the-Middle (MITM) attacks. Each signature is unique and connected to a specific handshake due to the incorporation of nonces in the transcript, preventing replay attacks, where an attacker captures a signature of a previous session and reuses it in another one.

3.3.2.2 Implementation & Analysis

While the theory of dual signatures was introduced in Section 3.2.5, its practical utility lies in the signing of a dynamic Session Transcript. Instead of signing static identification data, the protocol signs a hash of all ephemeral parameters exchanged up to that point.

In practice, this transcript includes the ephemeral public keys from both the X25519 and Kyber exchanges, along with the nonces generated by the client and the server. This achieves Identity Binding, as the server's long-term keys effectively "seal" the temporary keys of the session.

$$\begin{aligned}\sigma_{ECDSA} &= \text{Sign}_{ECDSA}(sk_{ECDSA}, T), \\ \sigma_{ML-DSA} &= \text{Sign}_{ML-DSA}(sk_{ML-DSA}, T)\end{aligned}$$

Once generated, both signatures are concatenated into a composite authentication payload and transmitted to the client:

$$M = \{pk_{ECDSA}, pk_{ML-DSA}, N_S, \sigma_{ECDSA}, \sigma_{ML-DSA}\}$$

The client then deconstructs this payload and verifies each signature separately using the server's public keys. If an active attacker attempts a Man-in-the-Middle attack by replacing the Kyber public keys, the transcript hash will inevitably change, causing the signatures to fail verification. Furthermore, the inclusion of nonces ensures that every signature is unique to its specific session, preventing replay attacks where a signature from a previous handshake is reused.

3.3.3 Authenticated Hybrid Handshake Protocol

The final step in the construction of the system is the unification of the key exchange and authentication mechanisms into a single flow. The Authenticated Hybrid Handshake Protocol is the result of integrating the components described in the previous sections to establish a communication channel that is both private and authenticated. This protocol ensures that the session keys are not only secure against quantum adversaries but also cryptographically tied to the verified identity of the server.

3.3.3.1 Protocol Design & Flow

The final construction of the protocol integrates all the previously defined building blocks into a single, cohesive exchange. As previously noted, symmetric primitives like HMAC cannot be used for this initial handshake phase since no shared secret has been derived yet, and they do not guarantee non-repudiation. To address this issue, the hybrid digital signature mechanism described in the previous sections is implemented as the core of the authentication process. Following the Defense-in-Depth property, the server's identity is authenticated only if both signature schemes (ECDSA and ML-DSA) verify successfully. This ensures that only if both schemes are simultaneously broken can an attacker forge authentication.

As analysed in Section 3.3.1.2, the raw outputs of the asymmetric channels (ss_{X25519} and ss_{ML-KEM}) are combined at the symmetric layer to build the IKM:

$$IKM = ss_{X25519} || ss_{ML-KEM}$$

which is passed to the extraction engine to clear algebraic dependencies and produce a uniform Pseudorandom Key (PRK):

$$PRK = \text{HKDF.Extract}(\text{Salt}, IKM)$$

To complete this key derivation pipeline and bridge the gap with the active cryptographic state machine, the final binding phase relies on the step-by-step serialization and expansion of this entropy:

1. **Serialization:** The shared secrets are converted to a byte sequence. This step is mandatory because the subsequent cryptographic functions operate on byte strings, and it ensures both Client and Server agree on the exact binary format.
2. **Expansion (HKDF-Expand):** Finally, **hkdf_expand** function projects this entropy into the final 32-byte session key:

$$K_{\text{session}} = \text{HKDF.Expand}(\text{PRK}, \text{Info}, 32).$$

By using this pipeline, it is ensured that the final symmetric key is cryptographically strong, effectively bridging the gap between raw, non-uniform output of X25519 and ML-KEM and the uniform requirements of symmetric encryption mechanisms such as AES.

To verify the end-to-end correctness of the protocol, a full simulation of the handshake was performed. The test environment monitors the transition between states from the initial key generation to the final key confirmation, ensuring that the modular components (KEMs and Signatures) interact without errors.

3.3.3.2 Security Analysis

The security of the proposed hybrid key exchange is based on the independence of its components and the properties of the underlying key derivation function. An adversary capable of breaking classical Diffie–Hellman using a quantum computer would still be unable to compute the ML-KEM shared secret. Conversely, any unforeseen weakness in ML-KEM would not compromise the classical Diffie–Hellman secret.

Because the final session key is derived from both secrets, an attacker must break both mechanisms to recover it. This aligns with the standard security intuition behind hybrid cryptographic constructions and reflects current best practices recommended during the post-quantum transition period.

Additionally, forward secrecy is preserved with respect to the classical component by using ephemeral Diffie–Hellman keys. Even if long-term keys are compromised in the future, past session keys remain protected as long as at least one component remains secure.

Practical security is further reinforced by authenticated handshake mechanisms. The inclusion of ML-DSA-44 signatures ensures that any active man-in-the-middle attempts are detectable. Since the verification pipeline enforces a strict logical conjunction, an adversary is prevented from injecting or altering key exchange messages without simultaneously compromising both cryptographic structures; a failure in either signature scheme immediately aborts the connection. Furthermore, the integrity of the handshake transcript is strictly maintained via length-prefixed encoding and HMAC-SHA256 verification, ensuring that any unauthorized modification to the exchanged values results in an immediate handshake failure.

Finally, the hybrid construction provides a layered defense: even if one cryptographic primitive is weakened in the future, the session key remains secure due to the remaining strong component. This design embodies both post-quantum readiness and classical forward secrecy, forming a robust foundation for secure session key establishment. The end-to-end correctness of this cryptographic design, along with its execution logs and state-machine transitions under simulation, has been validated through a test script which is detailed in Appendix C.5.

3.4 Benchmark and Measurements

Once the theoretical foundations and structural integration of the hybrid primitives have been established, this section evaluates their performance through empirical benchmarking. The analysis focuses on three main operational objectives:

- **Identifying Network Overheads and Bottlenecks:** Evaluating how the size of implemented PQC primitives (specifically ML-KEM-512 and ML-DSA-44) affects the total handshake size. The focus is on the Maximum Transmission Unit (MTU), which establishes the standard capacity limit for a single network packet (approximately 1500 bytes).
- **Validating the Technical Viability:** Demonstrating that the implemented hybrid handshake protocol can be used in real world applications. This will be done by determining that the total time to complete the handshake is reasonable for a web or an application.

- Analysing the trade-off between Security and Performance: Determining if the increment on byte size when transitioning from classical cryptography to post-quantum and hybrid schemes justifies the security level of the protocol.

Unlike classical ECC primitives such as X25519 or ECDSA, whose cryptographic payload easily fits within a single IP packet, PQC algorithms (particularly those based on lattices) produce significantly larger signatures. If the total handshake size exceeds the 1,500-byte threshold, the network protocol is forced to perform IP Fragmentation (more than one IP packet required). This fragmentation splits the cryptographic data into multiple packets, which can lead to increased real-world latency, higher risks of packet loss, and potential issues with firewalls or routers that may drop fragmented traffic.

To evaluate the implications commented above, four scenarios were benchmarked:

1. Classical: X25519 + ECDSA P-256.
2. Post-Quantum: ML-KEM-512 + ML-DSA-44.
3. Hybrid KEM (without ML-DSA): (X25519 + ML-KEM-512) + ECDSA P-256.
4. Full Hybrid: (X25519 + ML-KEM-512) + (ECDSA P-256 + ML-DSA-44).

3.4.1. Analysis of Results

After executing the performance tests over the four scenarios previously defined, these are the results obtained:

- A. **Network Payload and IP Fragmentation Analysis:** In this test it is analysed the total number of data exchanged in the whole handshake, which is a key factor for network efficiency and performance, more specifically regarding the MTU, as commented before.

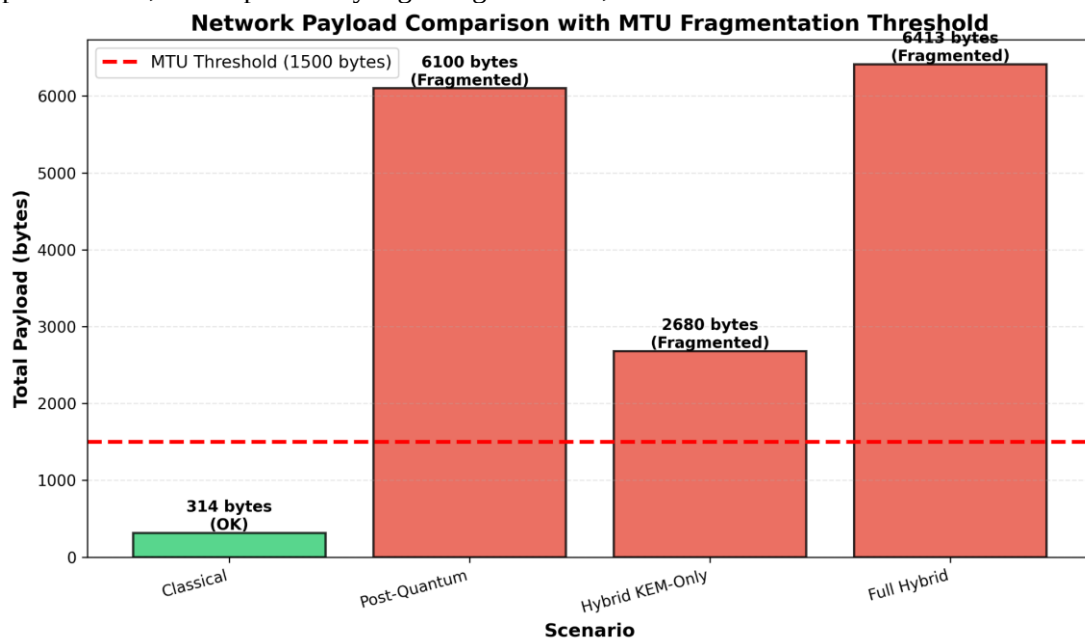


Figure 1: Network payload and IP fragmentation across the four evaluated handshake scenarios.

As illustrated in Figure 1, the classical scenario is the only one that operates in a single IP packet (314 bytes), while the Full Hybrid scenario reaches a total of 6413 bytes, which forces the fragmentation into 5 IP packets.

A significant finding is that the Hybrid KEM reduces network overhead by 60% compared to the Full Hybrid model, significantly decreasing the risk of packet loss and reassembly issues in firewalls, even though it fragments into 2 IP packets (since it reaches 2680 bytes).

B. Latency Comparison and Statistical Stability: The main goal of this test is measuring the total computation time (latency) of the complete handshake protocol once the necessary cryptographic libraries have been initialized by the system. Also, both the standard deviation (σ) and the coefficient of variation (CV) are measured with the purpose of analyzing the statistical stability and predictability of each scenario.

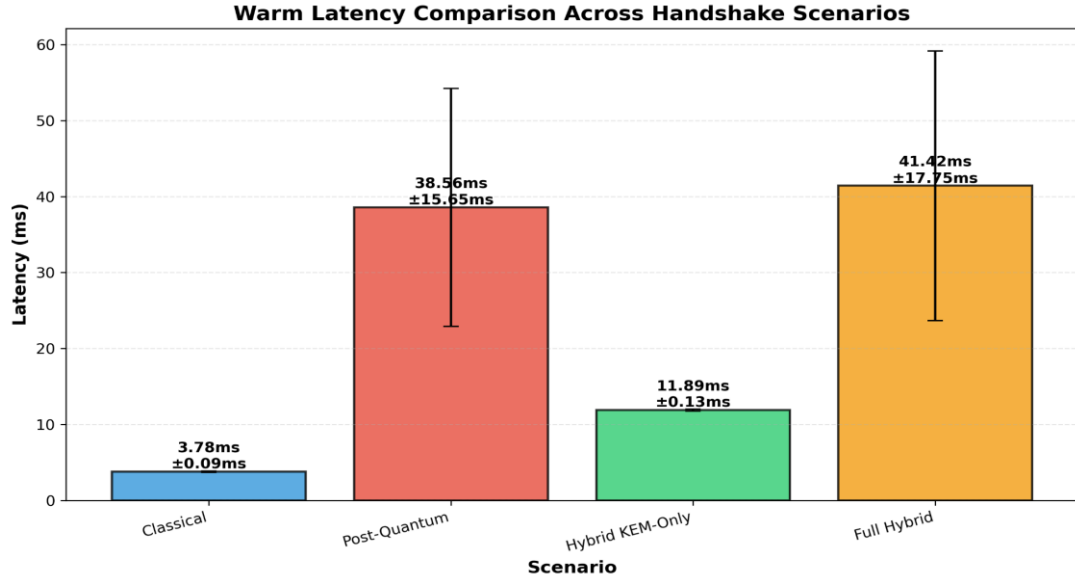


Figure 2: Total computational latency and statistical dispersion (σ) across the four evaluated handshake scenarios.

Figure 2 clearly shows how both the latency and the variance of the Full Hybrid Scenario represents a considerable increase compared to the Classical protocol.

It is particularly interesting to note that while the classical protocol is very stable ($\sigma = \pm 0.09 \text{ ms}$, $CV = 2.4\%$), both the Post-Quantum ($\sigma = \pm 15.65 \text{ ms}$, $CV = 40.6\%$) and the Full Hybrid ($\sigma = \pm 17.75 \text{ ms}$, $CV = 41.2\%$) scenarios show the opposite. This indicates that post-quantum algorithms are much more sensitive to system resource fluctuations as memory management or cache misses, which can affect the Quality of Services (QoS) in concurrent servers or time-sensitive applications.

The main cause of this instability is the rejection sampling in the ML-DSA-44 (Dilithium) algorithm, applied during signature generation. In contrast with ECDSA, which produces signatures in constant time by using deterministic nonce derivation, ML-DSA generates signature candidates iteratively and tests whether they satisfy or not the rejection boundaries. The problem of this workflow is that every rejection requires repeating all the computation process with a new random seed, leading to this variable latency value.

C. Component Timing Breakdown: To understand how the computational resources are distributed across the different phases of the handshake, each scenario was decomposed into three different phases:

1. Key Generation: This phase comprises the generation of ephemeral keys by both the client and server, which include X25519 in the Classical and Hybrid KEM and, additionally, ML-KEM-512 in Post-Quantum and Full Hybrid scenarios.
2. Encapsulation/Signing: This phase measures the server-side operations required to secure the exchange, including both the shared secret encapsulation (ML-KEM-512)

- and the digital signature's generation (ML-DSA-44) to authenticate the server's identity.
3. Decapsulation/Verification: This phase covers the client-side operations, composed by signature verification, KEM decapsulation and shared secret computation.

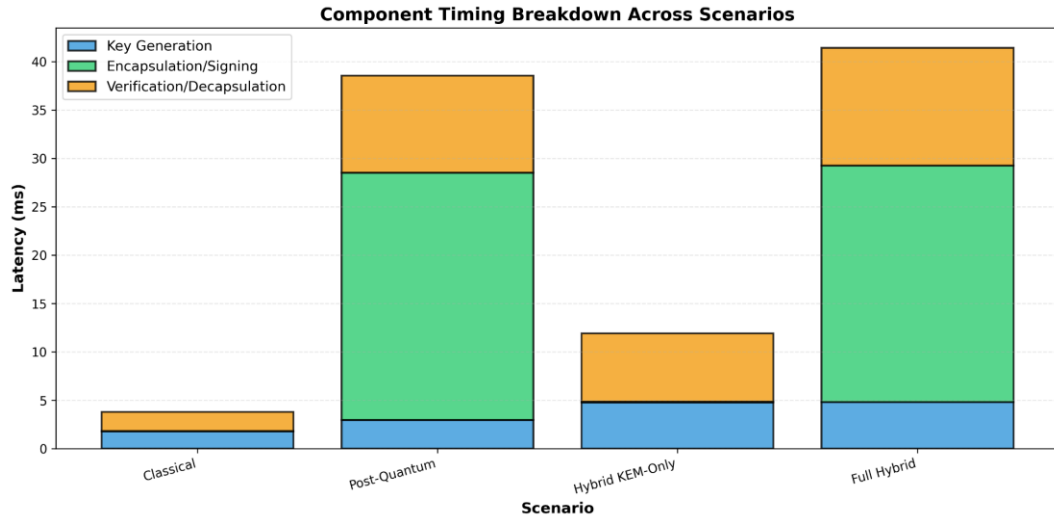


Figure 3: Functional timing breakdown of individual handshake phases across the evaluation profiles.

The results illustrated in Figure 3.3 clearly confirm what was explained in previous sections: the performance bottleneck of the scenarios where post-quantum algorithms are implemented is tied strictly to digital signatures. A detailed comparison of the Encapsulation/Signing phase reveals that while ECDSA signature generation only consumes 0.055-0.103ms (less than 0.3% of total latency), the integration of ML-DSA-44 (Dilithium) increases the signing latency over 25ms (95% of total latency). This demonstrates that the post-quantum digital signature is, by a wide margin, the most computationally expensive operation within the entire handshake protocol.

3.4.2 Synthesis and Technical Interpretation

The results obtained from these benchmarks highlight a critical engineering conflict between security and operational efficiency. While the Classical scenario offers the lowest latency, no protection against emerging quantum threats is provided. In contrast, Defense-in-Depth principle is achieved with the Full Hybrid scenario, but it leads to a 20x increase in total handshake payload compared to the Classical scenario, which leads into a mandatory IP fragmentation, increasing the probability of packet loss or rejection in worst case scenarios.

Table 2: Performance and Cryptographic Synthesis of the Evaluated Handshake Profiles.

Metric	Classical	Hybrid KEM	Post-Quantum	Full Hybrid
Cryptographic Strength	Vulnerable to Quantum Computation	Post-Quantum Resistant Confidentiality (Classical Authentication)	Lattice-Based	Defense-in-Depth
Handshake Size	314 bytes (1x)	2.6 kilobytes (8.5x)	6.1 kilobytes (19.4x)	6.4 kilobytes (20.4x)

IP Fragmentation	No, 1 packet	Yes, 2 packets	Yes, 5 packets	Yes, 5 packets
Average Latency	3.78 ms	11.89 ms	38.56 ms	41.42 ms
Latency Determinism	High ($\sigma = 0.09\text{ ms}$, $CV = 2.4\%$)	High ($\sigma = 0.13\text{ ms}$, $CV = 1.1\%$)	Low ($\sigma = 15.65\text{ ms}$, $CV = 40.6\%$)	Low ($\sigma = 17.75\text{ ms}$, $CV = 42.8\%$)

The collected data confirms that the transition to a quantum-resistant architecture not only represents a quantitative change in packet size, but a qualitative shift in protocol behaviour.

- **The Performance Tax of Authentication:** As commented before, comparing the Hybrid KEM-Only with the Full Hybrid scenario reveals that the primary bottleneck is not the lattice-based key exchange (ML-KEM), but the digital signature (ML-DSA). The latency increases by approximately 250% when ML-DSA is introduced, shifting the protocol from a deterministic state to a stochastic one.
- **Computational Jitter:** The high Coefficient of Variation ($CV > 40\%$) in the Post-Quantum and Full Hybrid scenarios is a direct consequence of the rejection sampling mechanism. This variability represents a critical factor for Quality of Service (QoS) in real-time systems, as the handshake duration is no longer a fixed cost but a variable distribution.
- **Network Viability:** The jump from 1 to 5 IP fragments in the Full Hybrid model significantly increases the risk profile for network-level failures. In restrictive environments or congested links, the necessity of reassembling five fragments to complete a single handshake could lead to a higher rate of connection timeouts compared to the more compact 2-packet Hybrid KEM model.

3.4.3 Practical Recommendations for Deployment

Based on the experimental results, the following guidelines are proposed for real-world cryptographic migration:

1. **Prioritize Hybrid KEM for Standard Traffic:** In applications where latency and network stability are critical (standard web browsing), the Hybrid KEM (without ML-DSA-44) model is the most balanced choice, since it secures data confidentiality against "Harvest Now, Decrypt Later" threats while avoiding the extreme overhead of post-quantum signatures as ML-DSA.
2. **Mandatory Full Hybrid for Long-Term Sensitivity:** For governmental or financial assets requiring both integrity and non-repudiation during decades, the Full Hybrid model must be mandatory. Although its latency is thought to be 10x higher than the Classical scenario, it remains well within acceptable thresholds for modern applications, where typical Round-Trip Times (RTT)—the total time it takes for a data packet to travel from the source to the destination and back—often exceed 50-100ms.
3. **Required Adaptation of Network Infrastructure:** Systems must be provisioned to handle MTU exceedance surpassing. Middleboxes like firewalls or load balancers to ensure they correctly support IP fragmentation and larger initial handshake frames without triggering DDoS protection mechanisms.

Conclusions & Future Work

This thesis provides a systematic assessment of the feasibility of migrating contemporary secure communications networks towards hybrid and post-quantum cryptographic schemes, considering both practical and computational implications. The study begins in Chapter 1 with a review of the algebraic foundations of classical public-key infrastructures, establishing the context for analysing the impact of the computational vulnerabilities of the integer factorization and discrete logarithm problems under Shor’s algorithm.

To address these vulnerabilities, Chapter 2 explores the theoretical foundations of lattice-based cryptography, detailing how multidimensional hardness problems such as Module-LWE and Module-SIS provide quantum resistance, while introducing the hybrid paradigm as a transitional mitigation strategy.

Moving from theory to implementation, Chapter 3 implements these concepts by constructing operational hybrid cryptographic building blocks and integrating them into a hybrid handshake protocol, followed by an empirical evaluation of their performance through extensive empirical benchmarking.

This study connects lattice-based constructions with practical protocol implementation for real-world network environments. The primary findings of this research can be summarized as follows:

- **Architectural Feasibility of Hybrid Key Exchanges:** In terms of viability and performance balance, the data collected during this study demonstrate that the hybrid KEM method described above (using X25519 and ML-KEM-512) constitutes a viable deployment strategy due to its performance characteristics. Specifically, this combination addresses the HNDL (Harvest Now, Decrypt Later) threat model for preserving confidentiality for data stored in the cloud for extended periods without introducing significant computational or protocol overhead on either computational resources or structural integrity of the handshake process.
- **Authentication Bottleneck:** A major contribution and finding of this research is the demonstration that there exists a direct correlation between the primary barriers to successful transition to post-quantum cryptography at present, i.e., primarily the computational overhead imposed by digital signatures and secondarily by key exchange mechanisms. The digital signature component is the primary factor causing significant increases in both computational cost and protocol overhead, and structural degradation to existing protocols during the migration, not the key exchange component. As demonstrated in the results section, integration of ML-DSA-44 (CRYSTALS-Dilithium) into the handshake sequence introduces an approximately $2.5 \times$ increase in average latency when compared to hybrid-KEM deployments alone. Furthermore, it was found that the majority—representing greater than 95 percent of the total time spent in the signing/encryption phases of the handshake sequence—can be attributed to the integration of the CRYSTALS-Dilithium digital signature scheme into the message processing flow.
- **Variability in Handshake Execution Time:** This study also identifies and demonstrates a significant qualitative shift and aspect of how the transition from deterministic classical handshakes to randomized; stochastic post-quantum routines affects protocol behaviour. It was observed through analysis of variance and other statistical metrics that the coefficient of variation (CV) of all fully post-quantum and hybrid implementations was greater than 40 percent. This high CV value represents a direct result of using the Fiat-Shamir with Abort framework. Due to the use of rejection sampling within this framework—applied to determine whether to accept or reject a proposed signature—handshake duration is now a random variable as opposed to being deterministically bounded. Each iteration results in either a valid signature

or a failure resulting in additional iterations until an acceptable signature is produced, transforming handshake times into probabilistic distributions instead of fixed values.

Based on both the performance and architectural trade-offs identified throughout the benchmarking phase, several purposes for future work are introduced. To improve the practical viability of these quantum-resistant frameworks in high-throughput networks, future research should focus on the following areas:

- **Evaluation of Alternative Post-Quantum Signatures:** Future work should implement and evaluate alternative lattice frameworks like Falcon-512 in a similar hybrid testbed to empirically determine if its compact size and different sampling methods justify the added architectural cost of implementation and specific mathematical complexities compared to ML-DSA.
- **Hardware Acceleration and Optimization:** Future work should assess the performance benefits of offloading polynomial matrix multiplications and hash functions onto dedicated hardware extensions or vector instructions, verifying that execution profiles are optimized to suppress the identified latency bottlenecks.
- **Impact of Multi-Tenant Concurrency Scenarios:** A natural extension of this research involves stress-testing the hybrid stack under heavy multi-tenant concurrent workloads, quantifying the impact of execution-time variability and computational overhead in testing environments.

Bibliography

Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, 124–134. <https://doi.org/10.1109/SFCS.1994.365700>

Shor, P. W. (1997). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5), 1484–1509. <https://doi.org/10.1137/S009753979529317X>

National Academies of Sciences, Engineering, and Medicine [NASEM]. (2019). *Quantum computing: Progress and prospects*. The National Academies Press. <https://doi.org/10.17226/25196>

Mavroeidis, V., Vishi, K., Zych, M. D. i Jøsang, A. (2018). The impact of quantum computing on present cryptography. *International Journal of Advanced Computer Science and Applications*, 9(3), 405–414. <https://doi.org/10.14569/IJACSA.2018.090354>

Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 212–219. <https://doi.org/10.1145/237814.237866>

National Institute of Standards and Technology [NIST]. (2016). *Submission requirements and evaluation criteria for the post-quantum cryptography standardization process*. U.S. Department of Commerce.

Brassard, G., Høyer, P. i Tapp, A. (1998). Quantum cryptanalysis of hash functions. *Lecture Notes in Computer Science*, (1443), 163–171. <https://doi.org/10.1007/BFb0055051>

Bindel, N., Brendel, J., Fischlin, M., Günther, F. i Stebila, D. (2019). Hybrid digital signature schemes. *Post-Quantum Cryptography*, 268–287. https://doi.org/10.1007/978-3-030-25510-7_15

Alagic, G., Alperin-Sheriff, J., Apon, D., Cooper, D., Dang, Q., Kelsey, J., Liu, Y.-K., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A. i Smith-Tone, D. (2022). *Status report on the third round of the NIST post-quantum cryptography standardization process* (NIST Internal Report 8413). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.IR.8413>

National Institute of Standards and Technology [NIST]. (2024c). *Post-Quantum Cryptography Standardization Project*. National Institute of Standards and Technology. Recuperat 24 de maig del 2026, des de <https://csrc.nist.gov/projects/post-quantum-cryptography>

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC Standard No. 8446). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8446>

National Institute of Standards and Technology [NIST]. (2024a). *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (Federal Information Processing Standards Publication FIPS 203). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.203>

Micciancio, D. i Goldwasser, S. (2002). *Complexity of lattice problems: a cryptographic perspective* (Vol. 671). Springer Science & Business Media.

Peikert, C. (2016). A decade of lattice cryptography. *Foundations and Trends® in Theoretical Computer Science*, 10(3–4), 283–424. <https://doi.org/10.1561/04000000074>

Hoffstein, J., Pipher, J. i Silverman, J. H. (2008). *An introduction to mathematical cryptography* (Vol. 1). Springer.

Goldreich, O., Goldwasser, S. i Halevi, S. (1997). Public-key cryptosystems from lattice reduction problems. *Advances in Cryptology—CRYPTO'97*, 112–131. Springer. <https://doi.org/10.1007/BFb0052231>

Babai, L. (1986). On Lovász' lattice reduction and the nearest vector problem. *Combinatorica*, 6(1), 1–13. <https://doi.org/10.1007/BF02579403>

Micciancio, D. i Regev, O. (2007). Worst-case to average-case reductions based on Gaussian measures. *SIAM Journal on Computing*, 37(1), 267–302. <https://doi.org/10.1137/050624952>

Micciancio, D. (2001). The shortest vector in a lattice is hard to approximate to within some constant. *SIAM Journal on Computing*, 30(6), 2008–2035. <https://doi.org/10.1137/S009753979935140X>

Regev, O. (2005). On lattices, learning with errors, random linear codes, and cryptography. *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*, 84–93. ACM. <https://doi.org/10.1145/1060590.1060603>

National Institute of Standards and Technology [NIST]. (2024b). *Module-Lattice-Based Digital Signature Standard* (Federal Information Processing Standards Publication FIPS 204). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.204>

Ajtai, M. (1996). Generating hard instances of lattice problems. *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1–9. ACM. <https://doi.org/10.1145/237814.237838>

Langlois, A. i Stehlé, D. (2015). Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3), 565–599. <https://doi.org/10.1007/s10623-014-9938-4>

Lyubashevsky, V., Peikert, C. i Regev, O. (2013). On ideal lattices and learning with errors over rings. *Journal of the ACM (JACM)*, 60(6), 1–35. <https://doi.org/10.1145/2535925>

Lyubashevsky, V., Peikert, C. i Regev, O. (2010). On ideal lattices and learning with errors over rings. *Advances in Cryptology—EUROCRYPT 2010*, 1–23. Springer. https://doi.org/10.1007/978-3-642-13190-5_1

Peikert, C., Regev, O. i Stephens-Davidowitz, N. (2014). Pseudorandomness of ring-LWE for any ring and Gaussian ciphertexts. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 445–454. ACM. <https://doi.org/10.1145/2591796.2591843>

Appendix

Table of Contents

Appendix A: Classical Cryptographic Workflows.....	42
A.1 RSA Mathematical Workflow	42
A.2 Diffie-Hellman Key Exchange Workflow	42
A.3 ElGamal Encryption Workflow	43
A.4 Elliptic Curve Workflow (ECDH)	43
Appendix B: Algebraic Proofs of Correctness.....	43
B.1 Algebraic Proof of Correctness for ML-KEM Verification.....	43
B.2 Algebraic Proof of Correctness for ML-DSA Verification	44
B.3 Algebraic Proof of Correctness for Falcon Verification.....	45
Appendix C: Implementation Details and Testing Traces.....	46
C.1 SHA-256 Algorithm Steps.....	46
C.2 Bit Clamping.....	46
C.3 Hybrid KEM Exchange Verification Test	46
C.4 Hybrid Authentication Verification Test	47
C.5 Authenticated Hybrid Handshake Protocol Verification Test	48

Appendix A: Classical Cryptographic Workflows

A.1 RSA Mathematical Workflow

The detailed workflow of RSA is described below:

- Key Generation:

1. Two uniformly random prime numbers p, q are chosen with bitlength $\lambda/2$, where λ is the security parameter
2. The modulus is computed as:

$$N = p \cdot q$$

3. Euler's totient function is used to compute the security parameters:

$$\varphi(N) = (p - 1)(q - 1)$$

4. A public exponent e is chosen such that $1 < e < \varphi(N)$ and $\gcd(e, \varphi(N)) = 1$
5. The private exponent d is computed as:

$$d \equiv e^{-1}(\text{mod } \varphi(N))$$

The public key is composed of (e, N) and the private key is composed of (d, N) .

- Encryption and Decryption:

- To encrypt a message m subject to the constraint $m < N$, the ciphertext is computed by the sender following the formula:

$$c \equiv m^e \text{ mod } N$$

- To decrypt those ciphertext c and recover the original message m , the receiver computes:

$$m \equiv c^d \text{ mod } N$$

- Digital Signatures:

- RSA is also used for signatures, which verify the authenticity of messages. A signature S can be created by the owner by computing:

$$S \equiv m^d \text{ mod } N$$

Finally, the verification can be performed by any entity to ensure that:

$$m \equiv S^e \text{ mod } N$$

A.2 Diffie-Hellman Key Exchange Workflow

The workflow of DH is defined over a cyclic group $(G, *)$ of order q and it is divided into three main parts:

1. Public parameters: A cyclic group G of order q and a generator g of G are chosen. These parameters are public.
2. Key Generation: Alice generates a random private exponent $a \in \mathbb{Z}_q$ (or $a \in \mathbb{Z}_{q-1}$, depending on the group's order) and computes the group element:

$$A = g^a$$

Bob generates a private exponent $b \in \mathbb{Z}_q$ and computes the group element:

$$B = g^b$$

These values are their public keys.

3. Shared Secret: Now both parties exchange their public values A and B . Alice uses Bob's public key and her private key to compute the shared secret as:

$$K = B^a$$

while Bob computes the shared secret by repeating the same process as Alice:

$$K = A^b$$

Both parties get the same shared secret due to the algebraic properties of cyclic groups:

$$K = (g^b)^a = (g^a)^b = g^{ab}$$

A.3 ElGamal Encryption Workflow

The ElGamal asymmetric encryption scheme consists of three distinct stages:

1. Key generation: A cyclic group G of prime order q and a generator g of the group G are chosen. The receiver selects a random secret key $x \in \mathbb{Z}_q^*$ and computes its public key:

$$h = g^x$$

Then the public key is composed by (G, q, g, h) and the secret key is defined exclusively by x .

2. Encryption: Given a message m , a random ephemeral key $r \in \mathbb{Z}_q^*$ is selected by the sender. The ciphertext is computed as:

$$(c_1, c_2) = (g^r, m \cdot h^r)$$

3. Decryption: Given the ciphertext (c_1, c_2) and the secret key x , the message is recovered by the receiver by computing:

$$m = c_2 \cdot (c_1^x)^{-1}$$

A.4 Elliptic Curve Workflow (ECDH)

The practical implementation of ECC involves three main steps and follows a similar structure to other DL schemes:

1. Domain Parameters Selection: A large prime integer p , curve coefficients a and b (of the Weierstrass equation defined above) and a base point or primitive element G are chosen. The point G generates a cyclic group of order n . These domain parameters are public and define the elliptic curve group used for all operations. To ensure security, standardized curves such as secp256r1 are used in practice.
2. Key Generation:
 - Alice picks a random secret key $a \in \{1, 2, \dots, n-1\}$, where n is the order of the base point G . Then she computes the corresponding public key by performing scalar multiplication:

$$A = aG$$

- Bob picks a random secret key $b \in \{1, 2, \dots, n-1\}$. Then he computes the corresponding public key:

$$B = bG$$

Then A and B are their public keys, which are points on the defined elliptic curve.

3. Shared Secret: Following the same logic of DH key exchange protocol, both parties exchange their public key over an insecure channel and perform the same operation:
 - Alice uses her private key to compute the secret point:

$$K = aB$$

- Bob computes the secret point as:

$$K = bA$$

Since $aB = a(bG) = abG$ and $bA = b(aG) = baG$, both parties establish the same shared secret K .

The security of ECC arises from the computational infeasibility of solving the ECDLP: given $Q = dG$, determining d is considered infeasible for sufficiently large p and n (Menezes et al., 1996; Koblitz, 1987).

Appendix B: Algebraic Proofs of Correctness

B.1 Algebraic Proof of Correctness for ML-KEM Verification

To prove the mathematical correctness of the ML-KEM (Kyber), it must be proven that the decapsulation process results in the successful recovery of the original ephemeral message m generated during encapsulation (National Institute of Standards and Technology [NIST], 2024).

In the Key Generation phase, the public key t is computed by using a random matrix $A \in R_q^{k \times k}$, a secret key vector $s \in R_q^k$ and the error vector $e \in R_q^k$ (Langlois & Stehlé, 2015):

$$t = A \cdot s + e \pmod{q}$$

During the Encapsulation phase, the sender generates both ciphertext components $u \in R_q^k$ and $v \in R_q$ by using the public key t and a seed message m (Bos et al., 2018; NIST, 2024):

$$\begin{aligned} u &= A^T \cdot r + e_1 \pmod{q} \\ v &= t^T \cdot r + e_2 + \left\lfloor \frac{q}{2} \right\rfloor \cdot m \pmod{q} \end{aligned}$$

where $e_1 \in R_q^k$ and $e_2 \in R_q$ are the error terms and $r \in R_q^k$ a random vector.

During Decapsulation, the receiver computes an evaluation polynomial $x \in R_q$ using the private key as a trapdoor (NIST, 2024):

$$x = v - s^T \cdot u \pmod{q}$$

To proof the correctness, the definitions of u and v must be substituted into the evaluation polynomial equation:

$$x = (t^T \cdot r + e_2 + \left\lfloor \frac{q}{2} \right\rfloor \cdot m) - s^T \cdot (A^T \cdot r + e_1) \pmod{q}$$

Substituting the public key t this equation expands into:

$$x = (s^T \cdot A^T + e^T) \cdot r + e_2 + \left\lfloor \frac{q}{2} \right\rfloor \cdot m - s^T \cdot A^T \cdot r - s^T \cdot e_1 \pmod{q}$$

Finally, applying both the distributive and associative properties of matrix-vector multiplication over the polynomial ring, the public terms cancel each other out and yields into:

$$\begin{aligned} x &= s^T \cdot A^T \cdot r + e^T \cdot r + e_2 + \left\lfloor \frac{q}{2} \right\rfloor \cdot m - s^T \cdot A^T \cdot r - s^T \cdot e_1 \pmod{q} \\ x &= \left\lfloor \frac{q}{2} \right\rfloor \cdot m + (e^T \cdot r + e_2 - s^T \cdot e_1) \pmod{q} \end{aligned}$$

As all the coefficients of secret and ephemeral error parameters are bounded by a centered binomial distribution, the total accumulated noise term satisfies the condition $\|e_{total}\| < q/4$ (Bos et al., 2018; Ducas et al., 2020). Because this total noise is extremely small and stays below the allowed threshold, the error is eliminated by a final mathematical rounding, successfully recovering the original message (Bos et al., 2018; NIST, 2024):

$$\left\lfloor \frac{2}{q} \cdot x \right\rfloor = m$$

B.2 Algebraic Proof of Correctness for ML-DSA Verification

To demonstrate the mathematical correctness of the mechanism explained in section 2.4.2.1.2, formulas of both the signature vector z and the public vector t must be substituted into the verifier's equation (National Institute of Standards and Technology [NIST], 2024b). Recall that the candidate signature vector z and the public key t are computed as follows (Ducas et al., 2018):

$$z = y + c \cdot s_1 \text{ and } t = A \cdot s_1 + s_2 \pmod{q}$$

Substituting these parameters into the verifier's equation for u it is obtained the following expanded polynomial equation:

$$u = A \cdot z - c \cdot t \pmod{q}$$

$$u = A \cdot (y + c \cdot s_1) - c \cdot (A \cdot s_1 + s_2) \pmod{q}$$

Applying the distributive property of matrix-vector multiplication over the polynomial ring, the equation expands to:

$$u = A \cdot y + A \cdot (c \cdot s_1) - c \cdot (A \cdot s_1) - c \cdot s_2 \pmod{q}$$

Because the cryptographic challenge c is a scalar polynomial within the ring, the matching deterministic secret terms cancel each other out:

$$u = A \cdot y + c \cdot A \cdot s_1 - c \cdot A \cdot s_1 - c \cdot s_2 \pmod{q}$$

This reduction simplifies the verification vector directly to:

$$u = A \cdot y - c \cdot s_2 \pmod{q}$$

Using the definition of the ephemeral public vector w , the relationship between the verifier's calculated vector u and the signer's vector w can be established as:

$$u = w - c \cdot s_2 \text{ mod } q$$

where the term $-c \cdot s_2$ is a perturbation vector. Because the coefficients of both the cryptographic challenge c and the secret error vector s_2 are small, their product is limited by the parameters:

$$\| -c \cdot s_2 \| \leq \beta$$

As the low-bits vector w_0 must satisfy the boundary condition:

$$\|w_0\| \leq \gamma_2 - \beta$$

the algebraic subtraction of the perturbation vector $-c \cdot s_2$ does not alter the higher-order bit allocation. The bound ensures that the geometric distortion introduced by the private key components is fully absorbed within the lower-order bits margin, avoiding any underflow or overflow carry bits into the high-bits plane (Lyubashevsky et al., 2020). Therefore, extracting the high-bits from the calculated evaluation vector u yields an algebraic array w_1' that is mathematically identical to the signer's original high-bits commitment vector:

$$w_1' = \text{Highbits}(u) = \text{Highbits}(w - c \cdot s_2) = w_1$$

This allows the verifier to recompute the cryptographic challenge verification hash:

$$c' = H(m \| w_1')$$

The signature satisfies the correctness proof only if the recomputed challenge matches the received challenge string exactly $c' = c$, confirming that the underlying linear combination has not been modified or corrupted by an unauthenticated adversary (Ducas et al., 2018; NIST, 2024b).

B.3 Algebraic Proof of Correctness for Falcon Verification

To demonstrate the mathematical correctness of Falcon's verification mechanism, the public key and signature parameters are substituted into the verifier's identity equation. First, it is necessary to recall that the key polynomials $f, g \in R$ and the public key $h \in R_q$ are related by the formula (Fouque et al., 2018):

$$h \equiv g \cdot f^{-1}(\text{mod } q)$$

During the signature process, the message m is hashed to the ring target $c = H(m) \in R_q$. The signer uses the private trapdoor to compute the short signature polynomials s_1 and s_2 such that:

$$s_1 = c - s_2 \cdot h \text{ (mod } q)$$

The receiver verifies the signature $z = (s_1, s_2)$ and recomputes $c = H(m)$. By substituting the definition of h into the previous equation it is obtained that:

$$s_1 + s_2 \cdot h \equiv (c - s_2 \cdot h) + s_2 \cdot h \text{ (mod } q)$$

Finally, applying the associative and distributive properties of polynomial multiplication (as in Dilithium's proof) the matching secret terms cancel out:

$$s_1 + s_2 \cdot h \equiv c - (s_2 \cdot h) + (s_2 \cdot h) \equiv c \text{ (mod } q)$$

While this cancellation proves that a signature derived from the true private key (trapdoor) collapses to the message digest c , it also introduces a vulnerability if left unchecked. Since this identity is evaluated modulo q , an adversary can choose an arbitrary polynomial $s_2' \in R_q$ and computing the matching polynomial:

$$s_1' \equiv c - s_2' \cdot h \text{ (mod } q)$$

This pair (s_1', s_2') satisfies the verification identity without requiring the possession of the private key (Prest, 2020).

To mitigate this risk, the correctness proof is derived from the Geometric Norm Validation. The signature satisfies the proof if and only if the combined vector simultaneously complies with the Euclidean boundary:

$$\|s_1, s_2\|^2 \leq \beta^2$$

where β is the threshold dictated by the Gaussian sampling density parameters.

Finding a polynomial pair that satisfies the linear combination stated before and falls within the short geometric bound β is equivalent to solving the Short Integer Solution (SIS) and Closest Vector Point (CVP) problems over lattices, which is computationally hard to achieve without possessing the private key (Ajtai, 1996; Micciancio & Regev, 2007). Thus, the geometric norm ensures that the cancellation could only have been performed by an entity possessing the cryptographic trapdoor.

Appendix C: Implementation Details and Testing Traces

C.1 SHA-256 Algorithm Steps

SHA-256 is a cryptographic hash function that takes an input of arbitrary length and produces a fixed 256-bit (32-byte) digest. The algorithm works through the following sequence:

1. **Message Padding:** It ensures that the message length is a multiple of 512 bits by padding it, which is done by appending a 1 bit, then enough 0 bits, and finally a 64-bit representation of the original message length.
2. **Variables Initialization:** It starts with eight 32-bit words initialized to specific constants derived from the fractional parts of the square roots of the first eight prime numbers.
3. **Block Expansion:** Each 512-bit block is expanded into 64 words of 32 bits each, using bitwise rotations and XOR operations.
4. **Compression Rounds:** For each block, the algorithm performs 64 rounds of compression, and each round involves bitwise operations and modular addition.
5. **Final Output Concatenation:** It concatenates the eight variables to produce the 256-bit output.

C.2 Bit Clamping

X25519 clamping involves setting specific bits of the 32-byte private key (clearing the lowest three bits and setting the highest bit) to prevent small-subgroup attacks and ensure the scalar is always a multiple of the curve's cofactor.

C.3 Hybrid KEM Exchange Verification Test

A verification test is performed to verify that the hybrid key exchange mechanism is successful and works as designed:

```
sergi@Air-de-Sergi tfg-practical % cd /Users/sergi/Documents/GitHub/tfg-practical && PYTHONPATH=. .venv/bin/python tests/
/test_hybrid_kem_exchange.py
=====
TEST: Hybrid Handshake (DH + Kyber)
=====

--- DH Key Generation (Static) ---
DH Public: fcd9ccceb5cc6524b46284e2a04a861a041cea59a6a9a5ec104df212114eef3f

--- Kyber Key Generation ---
Kyber Public: 49566b84766bb138169ff14832e4b967e649d36bb9da7824c10a5f4d94cd75ab... (truncated)

--- Client Side ---
Client DH key: 509cb04acdd132824519b5b30afacc9a3f0e8e12ce9ddb4bd7dc897b40aae8ce
Client Kyber secret: 542ef8fa070f4fa5a2485242b44d7f28094a11fc4e9a52b517aecaa526cc8bc3

--- Server Side ---
Server DH key: 509cb04acdd132824519b5b30afacc9a3f0e8e12ce9ddb4bd7dc897b40aae8ce
Server Kyber secret: 542ef8fa070f4fa5a2485242b44d7f28094a11fc4e9a52b517aecaa526cc8bc3

--- Hybrid Combination (DH || Kyber) ---
Final Client Key: 207d1790ffffbb29e6b7606c608bb90a72cb20404b8b1bb35ac692563aaee7452
Final Server Key: 207d1790ffffbb29e6b7606c608bb90a72cb20404b8b1bb35ac692563aaee7452
✓ Hybrid session keys match

--- HMAC Authentication (Post-Handshake) ---
✓ Hybrid HMAC verified

=====
HYBRID HANDSHAKE TEST PASSED!
=====
```

Figure 4: Terminal output demonstrating the successful execution and session key matching of the Hybrid Handshake verification test.

As demonstrated in the execution trace, the client/server symmetric keys generated demonstrate the functional accuracy of the hybrid KEM layers.

C.4 Hybrid Authentication Verification Test

A verification test is performed to authenticate the hybrid multi-algorithm authentication phase:

```
sergi@Air-de-Sergi ~ % cd /Users/sergi/Documents/GitHub/tfg-practical && PYTHONPATH=. .venv/bin/python tests/test_hybrid_signatures.py

=====
TEST: Classical ECDSA (P-256) Signatures
=====

[TEST 1] ECDSA Key Pair Generation
-----
✓ Generated ECDSA key pair
Public key (first 50 chars): b'-----BEGIN PUBLIC KEY-----\nMFkwEwYHKoZIzj0CAQYI...
Private key (first 50 chars): b'-----BEGIN PRIVATE KEY-----\nMIGHAgEAMBMGBqGSM4...

[TEST 2] ECDSA Sign and Verify (Valid Case)
-----
Message: Test handshake transcript for ECDSA
Signature (first 32 bytes): 3046022100a5008201121bf2e09abe25af4d9aad5097b191d2c7ae3a23b33f18...
✓ Signature verification: True

[TEST 3] ECDSA Verification with Modified Message
-----
Original message: Test handshake transcript for ECDSA
Modified message: Modified handshake transcript
× Verification result (should be False): False

[TEST 4] ECDSA Verification with Different Key
-----
Original signer vs. different public key
× Verification result (should be False): False

✓ ECDSA tests PASSED
```

Figure 5: Terminal output showing the key pair generation, valid signature verification, and error-handling tests for classical ECDSA (P-256) signatures.

```
=====
TEST: Post-Quantum Dilithium (ML-DSA-44) Signatures
=====

[TEST 1] Dilithium Key Pair Generation
-----
✓ Generated Dilithium key pair
Public key size: 1312 bytes
Private key size: 2560 bytes
Public key (first 32 bytes): 1487aa0f373062ef47d4567a66ca1142e2e4d112f3ef48c51efa0c0ca0633dc6...

[TEST 2] Dilithium Sign and Verify (Valid Case)
-----
Message: Test handshake transcript for Dilithium
Signature size: 2420 bytes
Signature (first 32 bytes): eb2320f128c3254431cd71e7e1e88245326b8ab9533d0d23559e88a6e581a227...
✓ Signature verification: True

[TEST 3] Dilithium Verification with Modified Message
-----
Original message: Test handshake transcript for Dilithium
Modified message: Modified handshake transcript
× Verification result (should be False): False

[TEST 4] Dilithium Verification with Different Key
-----
Original signer vs. different public key
× Verification result (should be False): False

✓ Dilithium tests PASSED
```

Figure 6: Terminal output showing the key pair generation, public/private key size details (in bytes), valid signature verification, and error-handling tests for Post-Quantum Dilithium (ML-DSA-44) signatures.

```

=====
TEST: Dual Signature Hybrid Authentication (ECDSA + Dilithium)
=====

[SETUP] Generate Dual Signature Keys
-----
✓ Generated ECDSA P-256 key pair
✓ Generated Dilithium ML-DSA-44 key pair

[TEST 1] Both Signatures Valid (Defense-in-Depth)
-----
    Handshake transcript: Client X25519: 0x1a2b3c... | Server X25519: 0x4d5e6f...

    ✓ ECDSA signature generated (72 bytes)
    ✓ Dilithium signature generated (2420 bytes)

    ECDSA verification: True
    Dilithium verification: True
    ✓ Hybrid authentication: True

[TEST 2] Attack: ECDSA Signature Corrupted
-----
    ECDSA verification (corrupted): False
    Dilithium verification: True
    ✗ Hybrid authentication: False
    ✓ Attack REJECTED (dual protection works!)

[TEST 3] Attack: Dilithium Signature Corrupted
-----
    ECDSA verification: True
    Dilithium verification (corrupted): False
    ✗ Hybrid authentication: False
    ✓ Attack REJECTED (dual protection works!)

    ✓ Hybrid authentication tests PASSED

=====
✓ ALL SIGNATURE TESTS PASSED!
=====

```

Figure 7: Terminal output of the Dual Signature Hybrid Authentication test suite, showing successful verification of classical and post-quantum signatures, along with the effective rejection of corrupted signature attacks.

As the test execution is successful it can be affirmed that the architecture satisfies three major requirements:

- Correctness: As client and server derive the same symmetric key.
- Hybrid Compatibility: Of classical and post-quantum KEMs.
- Secure Usability: Of the derived key for symmetric authentication.

C.5 Authenticated Hybrid Handshake Protocol Verification Test

A full simulation of the handshake was performed to verify the end-to-end correctness of the protocol. The test environment monitors the transition between states from the initial key generation to the final key confirmation, ensuring that the modular components (KEMs and Signatures) interact without errors. The complete runtime execution logs, console snapshots, and terminal outputs for this multi-layer handshake validation are compiled in the following test:

```
sergi@Air-de-Sergi: % cd /Users/sergi/Documents/GitHub/tfg-practical-8 && PYTHONPATH=. .venv/bin/python tests/test_complete_hybrid_authenticated_handshake.py

TEST: Protocol 3.4 - Hybrid Authenticated Handshake (Client/Server)
      With DUAL SIGNATURES: Classical (ECDSA) + Post-Quantum (Dilithium)
=====

[PHASE 0] Server Long-Term Key Setup (DUAL KEYS)
-----
✓ Server generated Dilithium (ML-DSA-44) signing key pair
  Public key (for PQ auth): 83dc4f7a02dbfe83c446b2b2c866998f62d5488154708f2bb9980478bc02e5b3... (truncated)
✓ Server generated ECDSA (P-256) signing key pair
  Public key (for classical auth): 2d2d2d2d2d242547494e20505424c4943284b45592d2d2d2d2d0a4d466b77457759484b6f5a497a6a3043415159494b6f5a... (truncated)

[PHASE 1] Client Ephemeral Key Generation
-----
✓ Client created with server's trusted DUAL public keys
✓ Client generated ephemeral keys
  Ephemeral DH public: 29b30ac527764e31cfff77e0933089a11cc213d80a57e327fa5984884bf7c66d
  Ephemeral Kyber public: b746184668af31740462a0a1a1e6ac18633a95c985eb3b8ab8b4ad759c107efa... (truncated)
→ Client sends to Server: (pk_dh_c, pk_kyber_c)

[PHASE 2] Server Ephemeral Key Generation & DUAL Signing
-----
✓ Server initialized with long-term DUAL signing keys
✓ Server generated ephemeral keys and signed with BOTH schemes
  Ephemeral DH public: f1381dadb71accb726a091dd737b31774e76138fa8cb11454b141a84a86595
  Ephemeral Kyber public: 97279a58ca8ab01b538a94d2e0a3c8580308384b9e1a81f2134d68b9f4073... (truncated)
  Dilithium signature: 70d63f03b566b348937ac377a0114a5f281eb7f26b3c5857fbf2bbf310f45d7... (truncated)
  ECDSA signature: 3045022108e37d6bb1e52d9f2da5ed3c85da4dd3c71a23409caf777291106d5a... (truncated)
→ Server sends to Client: (pk_dh_s, pk_kyber_s, sig_dilithium, sig_ecdsa)

[PHASE 3] Client DUAL Signature Verification
-----
✓ Dilithium (post-quantum) signature verified successfully
✓ ECDSA (classical) signature verified successfully
→ Client derived session key: 3ae09a7e41864a41ca49ebdf23585ca59c793c90fbca425bd1d5dcda440fd49
= Kyber ciphertext for server: 8488583345c1399d8015a234cf2aa9119786cd7aa8ccee143956462839e8e67... (truncated)
→ Client sends to Server: kyber_ciphertext

[PHASE 4] Server Shared Secret Derivation
-----
✓ Server computed shared secrets and derived session key
  Session key: 3ae09a7e41864a41ca49ebdf23585ca59c793c90fbca425bd1d5dcda440fd49
```

```
=====
VERIFICATION
=====
✓ Client and Server session keys MATCH
  Key: 3ae09a7e41864a41ca49ebdfdf23585ca59c793c90fbca425bd1d5dcda440fd49

[POST-HANDSHAKE] Message Authentication with HMAC-SHA256
=====
✓ HMAC tags match - message integrity verified
  Message: Authenticated message from client
  HMAC: ee0efdebfbac1ab54134e0e227769ef33a84899ded8ad93501216490396f80c
=====
✓ PROTOCOL 3.4 TEST PASSED!
=====

Summary:
- Authentication: DUAL SIGNATURES (classical ECDSA + post-quantum Dilithium)
- DH: X25519 (classical, forward-secret)
- PQ-KEM: ML-KEM-512 (post-quantum security)
- Session key: 32 bytes (hybrid DH || Kyber)
- Post-handshake: HMAC-SHA256 for message integrity

=====
TEST: Protocol 3.4 - MITM Attack Detection (DUAL SIGNATURES)
=====

[PHASE 1] Client generates and sends ephemeral keys
CLIENT → SERVER: (pk_dh_c, pk_kyber_c)

[PHASE 2] ATTACKER intercepts and substitutes own ephemeral keys
ATTACKER → CLIENT: (pk_dh_attacker, pk_kyber_attacker, sig_dilithium_attacker, sig_ecdsa_attacker)

[PHASE 3] Client verifies DUAL signatures
✓ DUAL signature verification FAILED (as expected)
  Reason: Handshake authentication failed: Dilithium (post-quantum) signature verification failed. Possible post-quantum MITM attack or corrupted PQ signature. Handshake aborted.
  Handshake aborted - MITM attack detected by BOTH authentication schemes!

=====
ALL PROTOCOL 3.4 TESTS PASSED!
=====
```

The deterministic execution logs demonstrate that the handshake phases progress sequentially, successfully establishing the secure channel without operational overhead or synchronization errors.

References:

- Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., & Stehlé, D. (2018). CRYSTALS - Kyber: A new lattice-based cryptographic function. In *Proceedings of the 2018 European Symposium on Security and Privacy (EuroS&P)* (pp. 353–367). IEEE. <https://doi.org/10.1107/EuroSP.2018.00032>
- Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., & Stehlé, D. (2020). *Crystals-Kyber* (Technical Report). National Institute of Standards and Technology.
- Langlois, A., & Stehlé, D. (2015). Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3), 565–599. <https://doi.org/10.1007/s10623-014-9938-4>
- National Institute of Standards and Technology. (2024). *Module-lattice-based key-encapsulation mechanism standard* (Federal Information Processing Standards Publication [FIPS] PUB 203). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.203>
- Ajtai, M. (1996). Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing* (pp. 1–9). ACM. <https://doi.org/10.1145/237814.237838>
- Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., & Stehlé, D. (2018). CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1), 238–268. <https://doi.org/10.13154/tches.v2018.i1.238-268>
- Fouque, P. A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., & Zhang, Z. (2018). *Falcon: Fast-Fourier lattice-based compact signatures over NTRU*. Technical Report, National Institute of Standards and Technology.
- Lyubashevsky, V., Nguyen, N. K., & Seiler, G. (2020). Practical lattice-based digital signature schemes. *Journal of Cryptology*, 33(4), 1–32. <https://doi.org/10.1007/s00145-020-09360-1>
- Micciancio, D., & Regev, O. (2007). Worst-case to average-case reductions based on Gaussian measures. *SIAM Journal on Computing*, 37(2), 358–402. <https://doi.org/10.1137/050635203>
- National Institute of Standards and Technology. (2024a). *Module-lattice-based digital signature standard* (Federal Information Processing Standards Publication [FIPS] PUB 204). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.204>
- National Institute of Standards and Technology. (2024b). *Asymmetric-key digital signature standard (Falcon)* (Federal Information Processing Standards Publication [FIPS] PUB 205). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.205>
- Prest, T. (2020). *Gaussian sampling on lattices and its applications to cryptography* (Doctoral dissertation, Université de Rennes 1).